

M. H. SABOO SIDDIK COLLEGE OF ENGINEERING

8, Saboo Siddik Polytechnic road, Byculla, Mumbai-8
Department of Computer Science and Engineering (AI & ML)

DATABASE MANAGEMENT SYSTEM

S.E. – CSE (AIML)

SEM- IV

COURSE CODE : 2164112

Dr. Muhib A. Lambay
Asst. Professor

Module 1: contd…

ER MODEL & Extended ER MODEL

Data Models

- Data Model is a logical structure of Database.
- Data model defines how data will be stored, accessed and updated in a database management system.
- Data model describes the design of database to reflect entities, attributes, relationship among data, constrains etc.
- A data model is an idea which describes how the data can be represented and accessed from software system after its complete implementation.
- It is a simple abstraction of complex real world data gathering environment.
- The main purpose of data model is to give an idea that how final system or software will look like after development is completed.

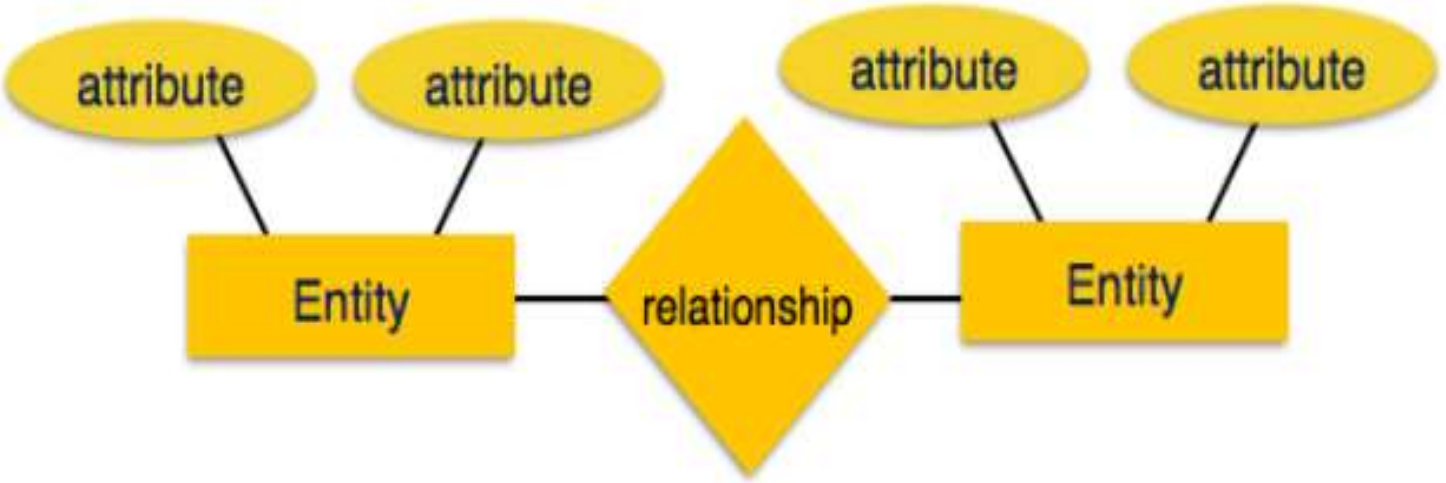
Entity-Relationship (ER) Model

- An Entity–relationship model (ER model) describes the structure of a database with the help of a diagram, which is known as Entity Relationship Diagram (ER Diagram).

- An ER model is a design of a database that can later be implemented as a database.

- ER Model is based on:
  - Entities and their attributes.
  - Relationships among entities.

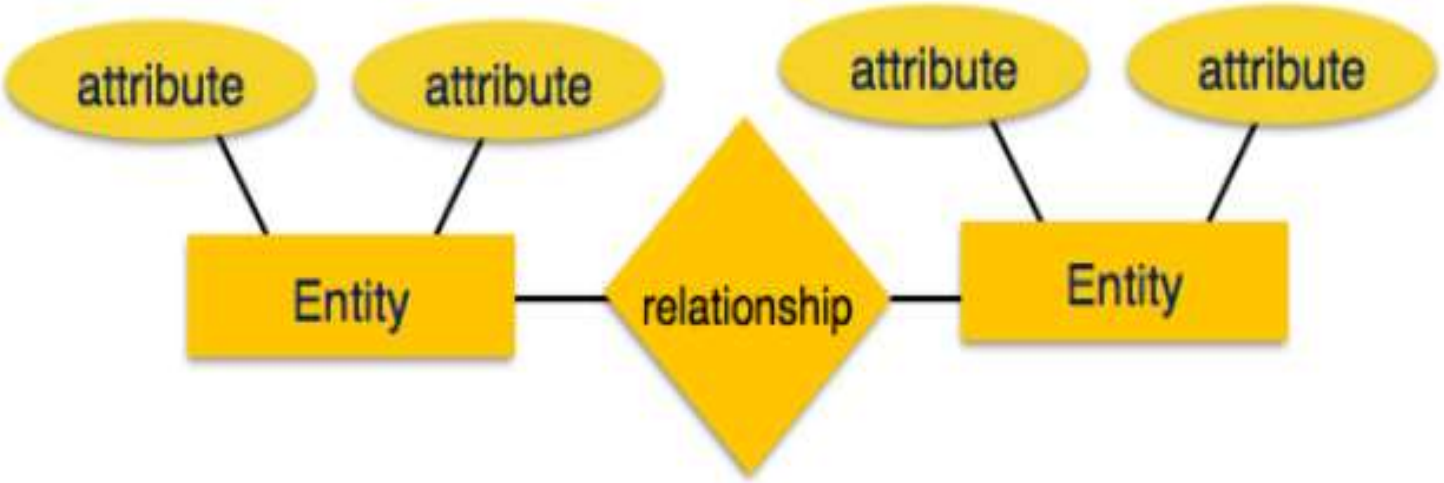
Entity-Relationship (ER) Model



•**Entity** – An entity in an ER Model is a real-world object having properties called **attributes**. Every **attribute** is defined by its set of values called **domain**. For example, in a school database, a student is considered as an entity. Student has various attributes like name, age, class, etc.

•An entity may be any object, class, person or place. In the ER diagram, an entity can be represented as rectangles.

Entity-Relationship (ER) Model



Relationship − A relationship is used to describe the relation between entities. Relationships are mapped with entities in various ways. Mapping cardinalities defines the relationship between two entities. **Diamond** is used to represent the relationship.

Mapping cardinalities −

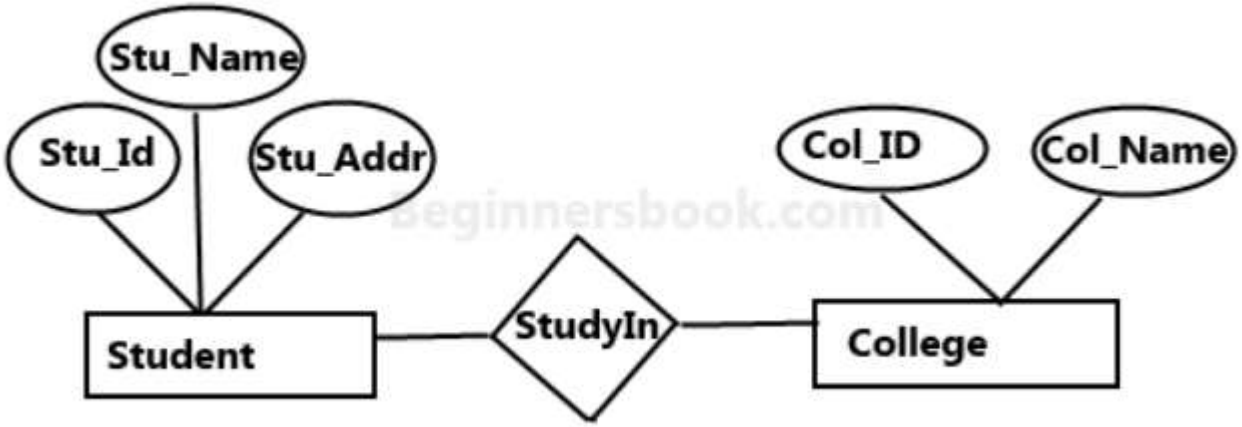
one to one

one to many

many to one

many to many

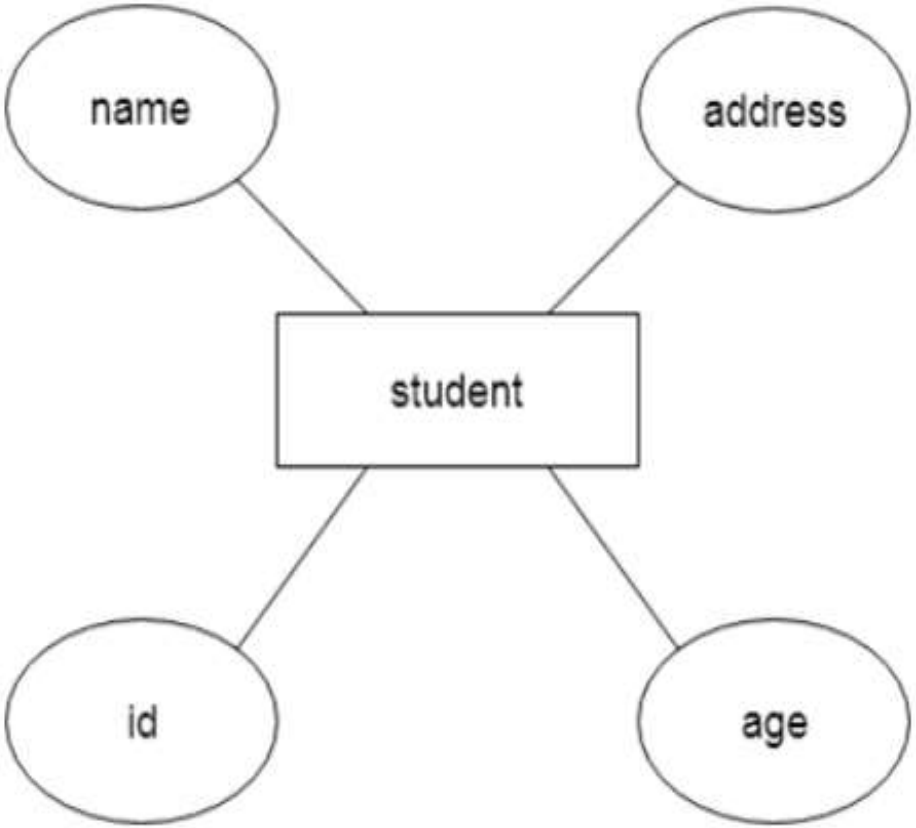
Entity-Relationship (ER) Model Example



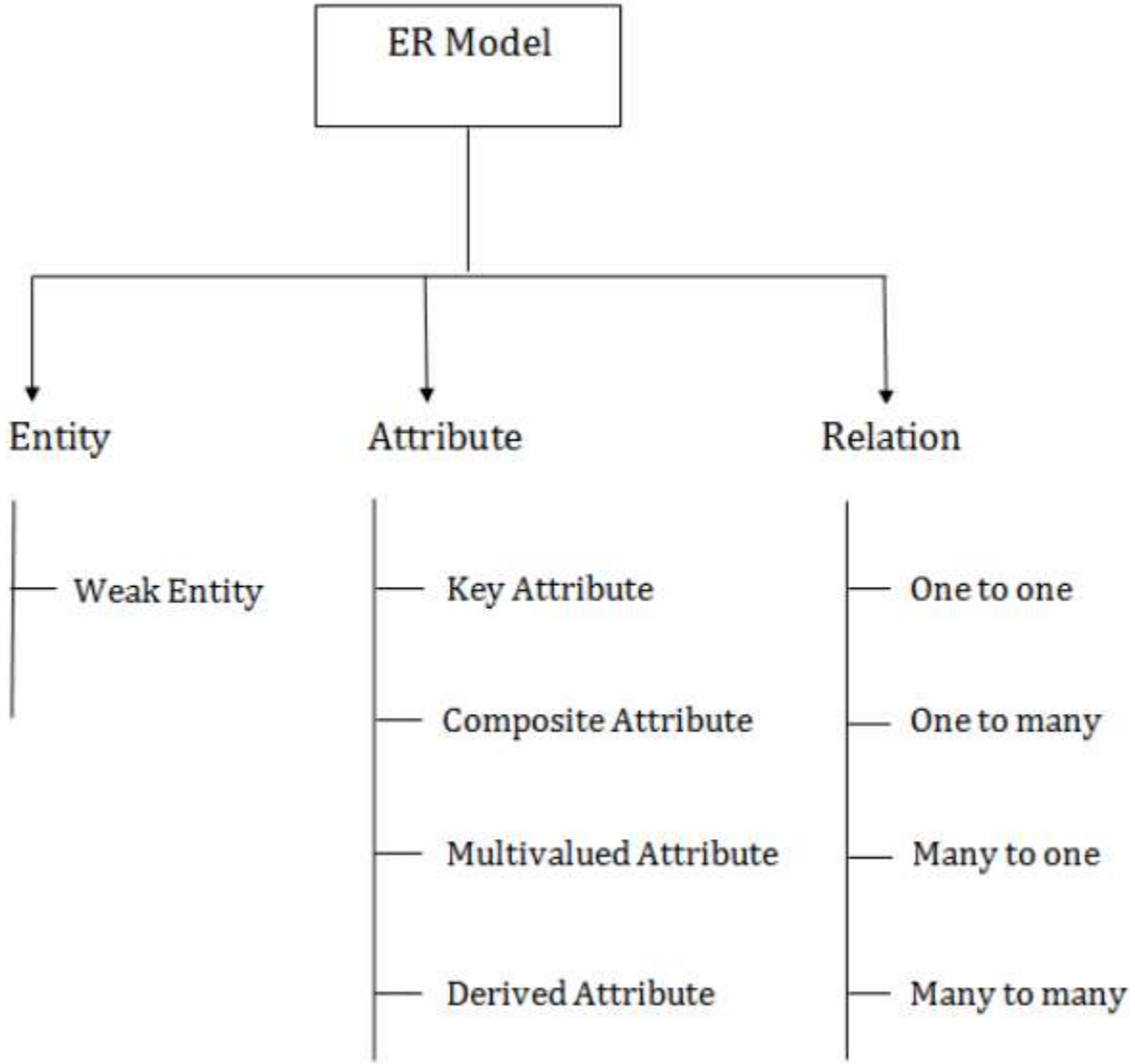
Rectangle: Represents Entity sets.
Ellipses: Attributes
Diamonds: Relationship Set
Lines: They link attributes to Entity Sets and Entity sets to Relationship Set

Entity-Relationship (ER) Model Example

For example, Suppose we design a school database. In this database, the student will be an entity with attributes like address, name, id, age, etc. The address can be another entity with attributes like city, street name, pin code, etc and there will be a relationship between them.

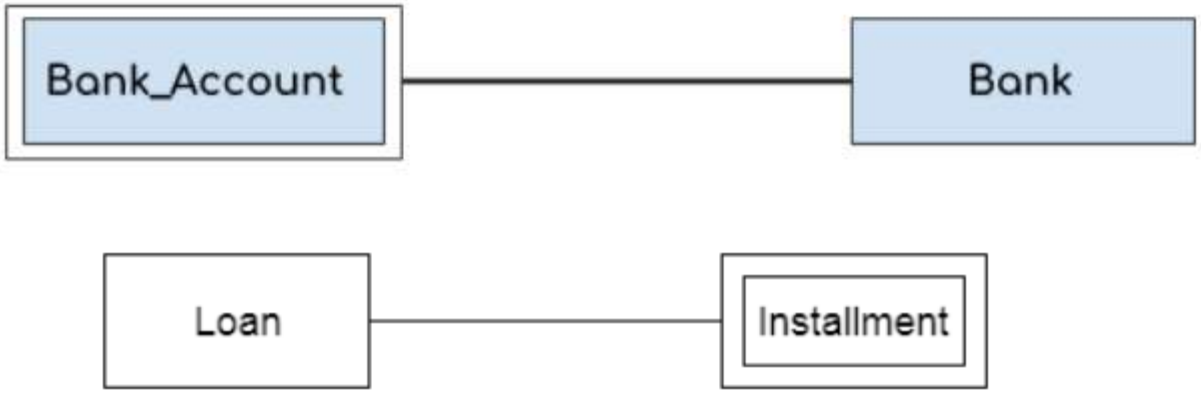


Components of ER Diagram



Weak Entity

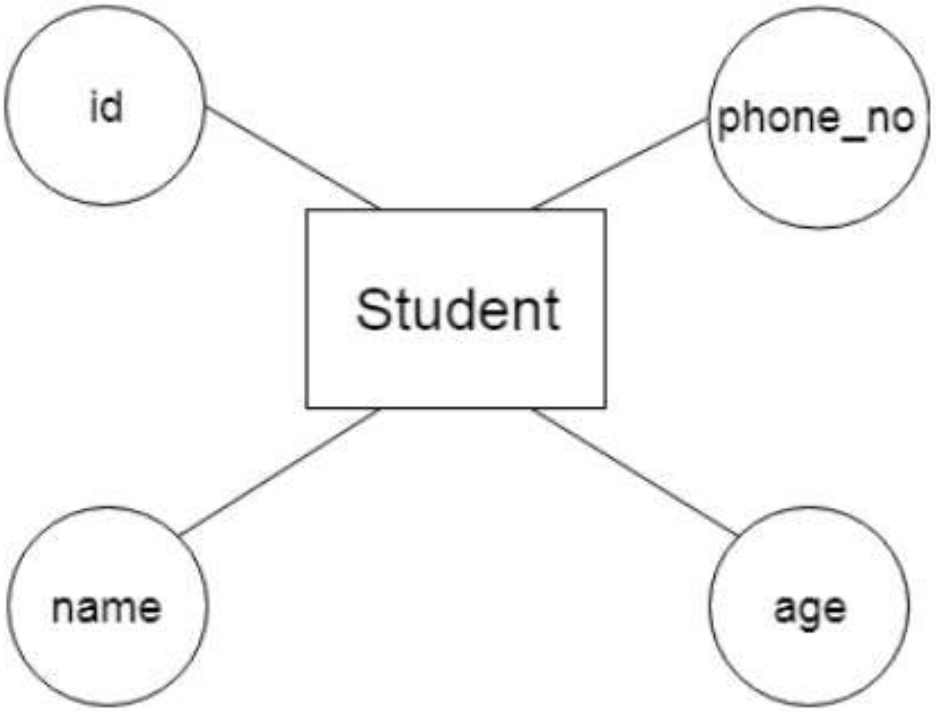
•An entity that cannot be uniquely identified by its own attributes and relies on the relationship with other entity is called weak entity. The weak entity is represented by a double rectangle. For example – a bank account cannot be uniquely identified without knowing the bank to which the account belongs, so bank account is a weak entity.



Attribute

•The attribute is used to describe the property of an entity. Eclipse is used to represent an attribute.

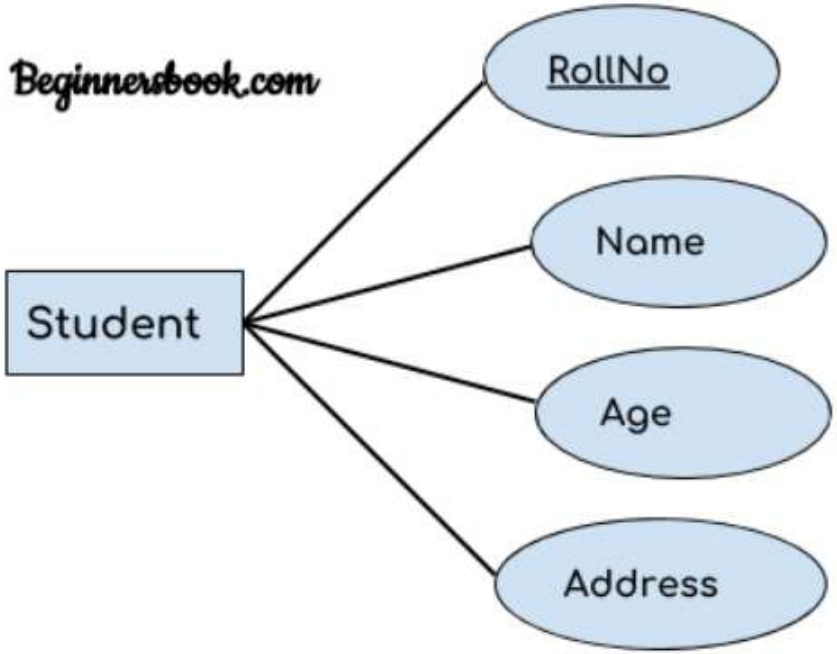
For example, id, age, contact number, name, etc. can be attributes of a student.



Key Attribute

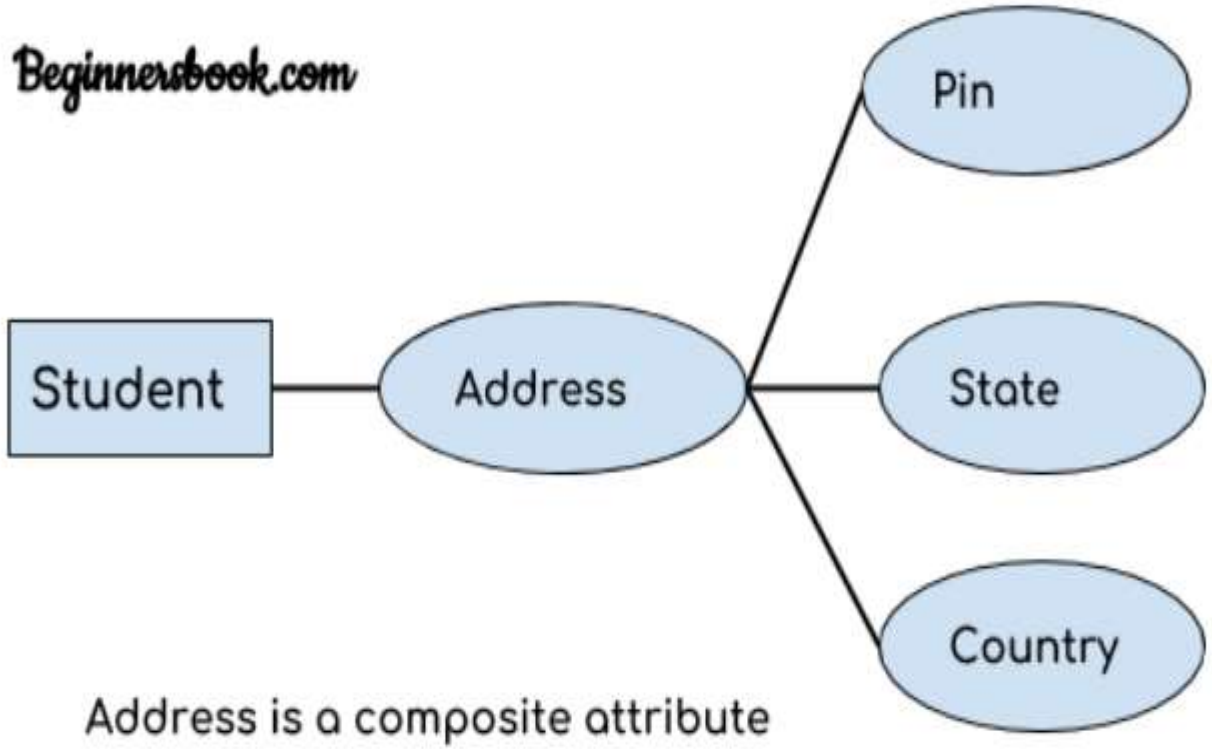
•A key attribute can uniquely identify an entity from an entity set. It represents a primary key. The key attribute is represented by an ellipse with the text underlined.

•For example, student roll number can uniquely identify a student from a set of students.



Composite Attribute

An attribute that is a combination of other attributes is known as composite attribute. For example, In student entity, the student address is a composite attribute as an address is composed of other attributes such as pin code, state, country.

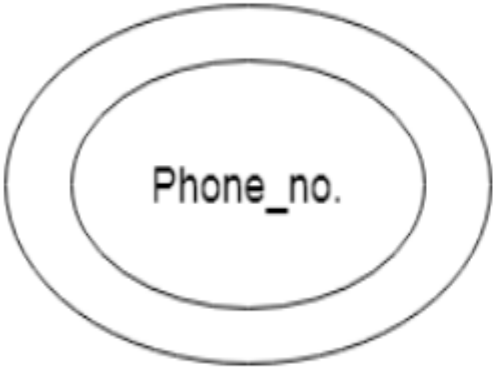


Address is a composite attribute

Multivalued Attribute

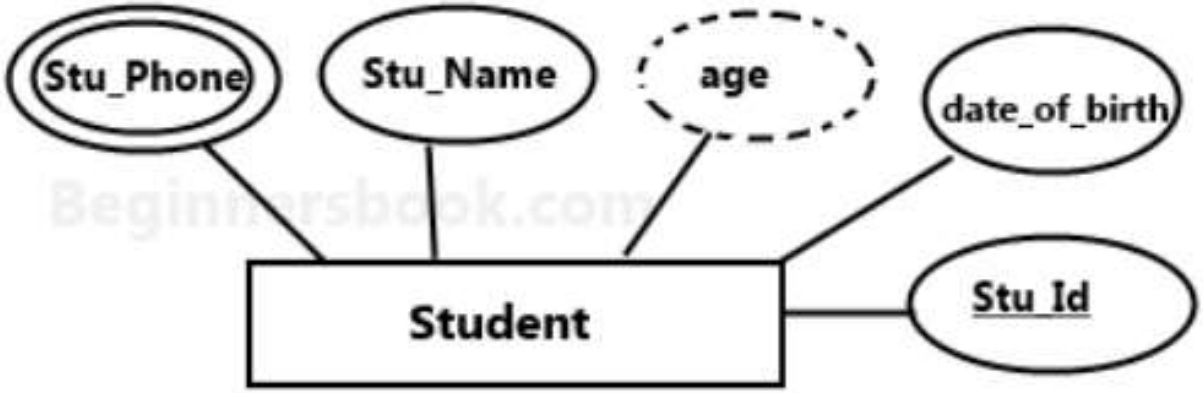
An attribute can have more than one value. These attributes are known as a multivalued attribute. The double oval is used to represent multivalued attribute.

For example, a student can have more than one phone number.



Derived Attribute

•A derived attribute is one whose value is dynamic and derived from another attribute. It is represented by dashed oval in an ER Diagram. For example – Person age is a derived attribute as it changes over time and can be derived from another attribute (Date of birth).



Relationship

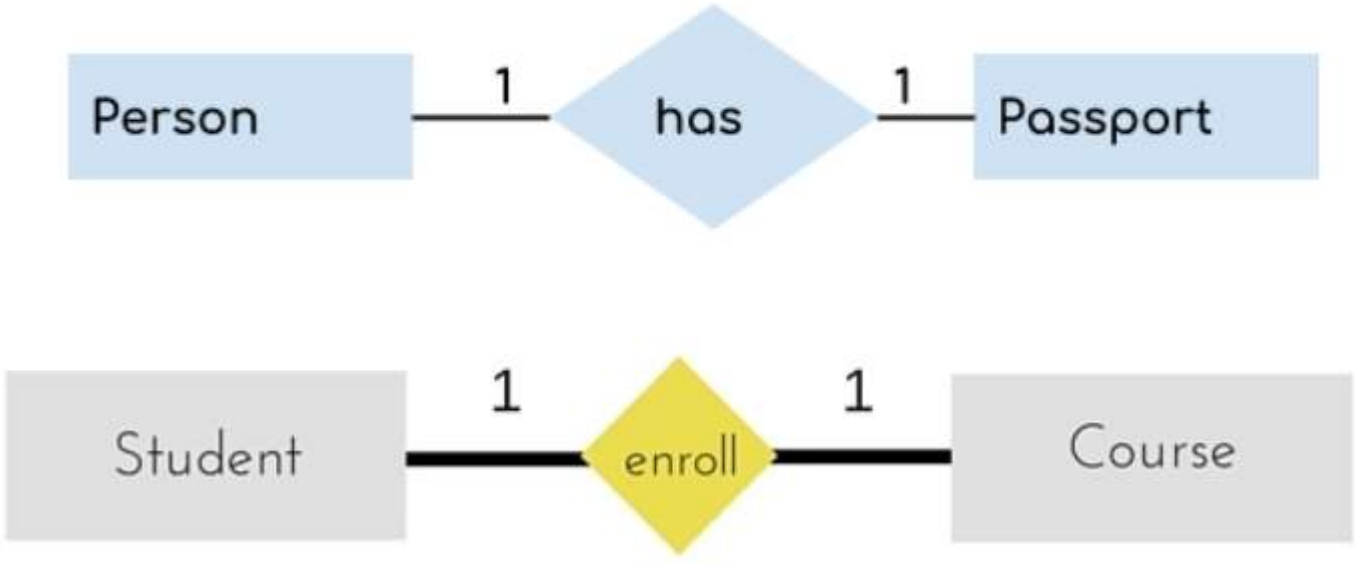
•A relationship is represented by diamond shape in ER diagram, it shows the relationship among entities.

•A relationship where two entities are participating is called a binary relationship. Cardinality is the number of instance of an entity from a relation that can be associated with the relation.



One to One Relationship

•When a single instance of an entity is associated with a single instance of another entity then it is called one to one relationship. For example, a person has only one passport.



Many-to-one relationship

When more than one instance of the entity on the left, and only one instance of an entity on the right associates with the relationship then it is known as a many-to-one relationship.

For example, Student enrolls for only one course, but a course can have many students.



For example – many students can study in a single college but a college can have many students at the same time.



Beginnersbook.com

Many-to-many relationship

When more than one instance of the entity on the left, and more than one instance of an entity on the right associates with the relationship then it is known as a many-to-many relationship.

For example, Employee can assign by many projects and project can have many employees.



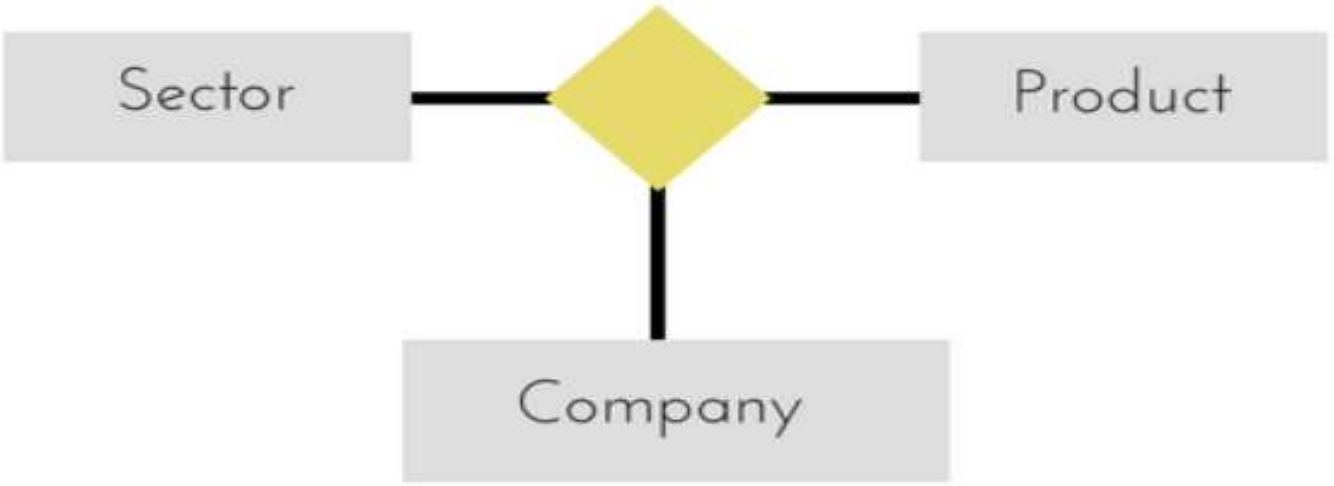
For example, Students as a group are associated with multiple faculty members, and faculty members can be associated with multiple students.



ER Diagram: Ternary Relationship

Relationship of degree three is called Ternary relationship.

A Ternary relationship involves three entities. In such relationships we always consider two entites together and then look upon the third.



- The above relationship involves 3 entities.
- Company operates in Sector, producing some Products.

Participation Constraints

1. Total Participation

- In Total Participation, every entity in the set is involved in some association of the relationship.
- It is indicated by a double line (═══) between entity and relationship.

For example: Every Department must have a Manager.



Fig. Total Participation

Suppose an entity set Student related to an entity set Course through Enrolled relationship set.
The participation of entity set course in enrolled relationship set is **partial** because a course may or may not have students enrolled in. It is possible that only some of the course entities are related to the student entity set through the enrolled relationship set.
The participation of entity set student in enrolled relationship set is **total** because every student is expect to relate at least one course through the enrolled relationship set.

2. Partial Participation

- In Partial Participation, not all entities in the set are involved in association of the relationship.
- It is indicated by a single line (———) between entity and relationship.

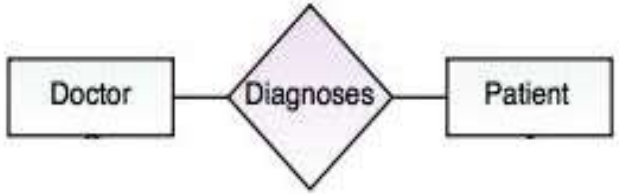


Fig. Partial Participation



Participation in Enrolled relationship set: **Partial** Course
Total Student

Participation Constraints

(2) **Partial Participation :** If not all entities in entity set E participate in relationships in relationship set R, the participation of E in R is termed as *Partial participation*.

Example : In *figure 2.13* the participation of CUSTOMER entity set in BORROWS relationship is partial as not all customers may have borrowed a LOAN.

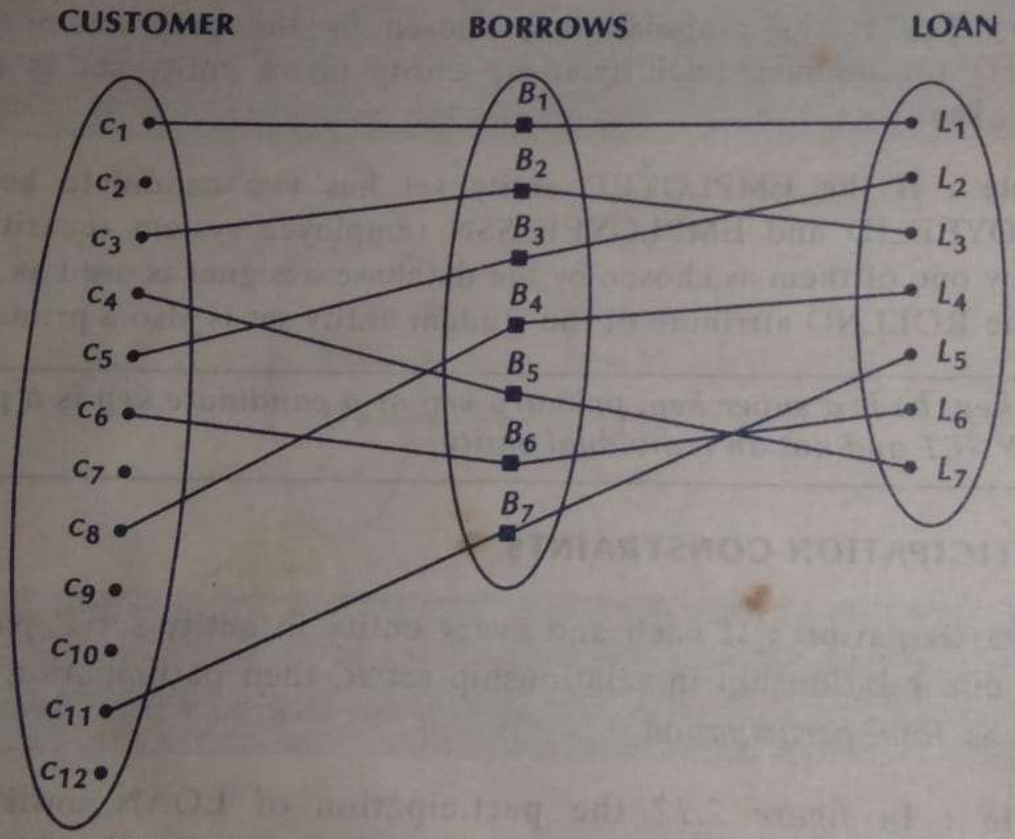
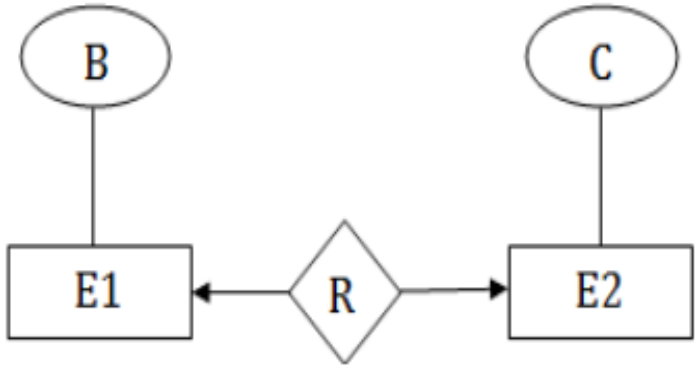


Fig. 2.13 : An instance of BORROWS relationship set.

Notation of ER diagram

One-to-one

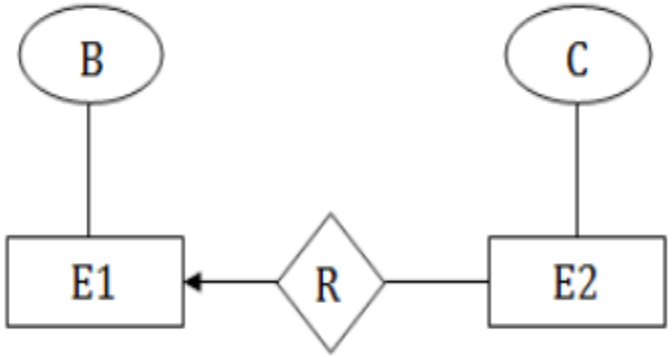
In one-to-one mapping, an entity in E1 is associated with at most one entity in E2, and an entity in E2 is associated with at most one entity in E1.



Notation of ER diagram

One-to-many

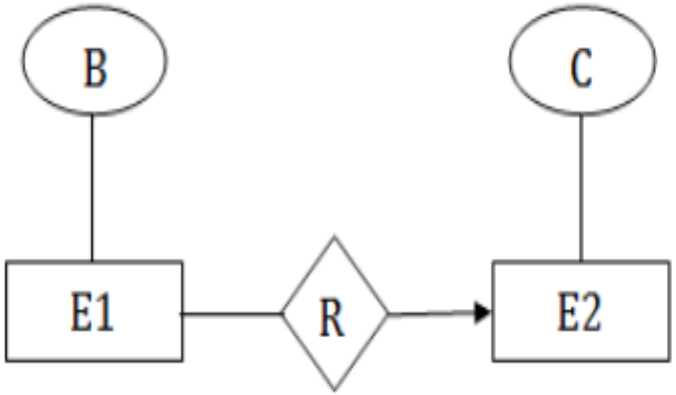
In one-to-many mapping, an entity in E1 is associated with any number of entities in E2, and an entity in E2 is associated with at most one entity in E1.



Notation of ER diagram

Many-to-one

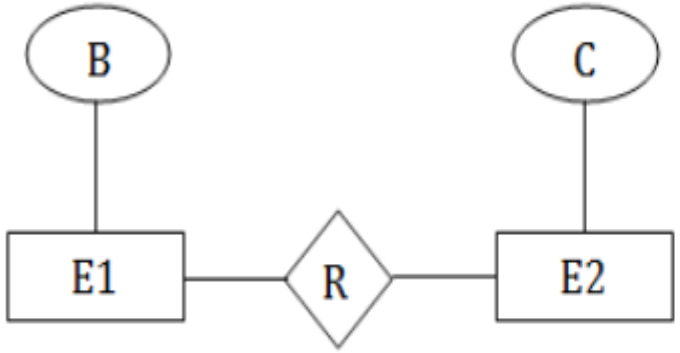
In one-to-many mapping, an entity in E1 is associated with at most one entity in E2, and an entity in E2 is associated with any number of entities in E1.



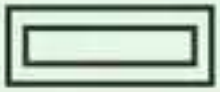
Notation of ER diagram

Many-to-many

In many-to-many mapping, an entity in E1 is associated with any number of entities in E2, and an entity in E2 is associated with any number of entities in E1.



Symbols Used in ER Diagrams

Figures	Symbols	Represents
Rectangle		Entities in ER Model
Ellipse		Attributes in ER Model
Diamond		Relationships among Entities
Line		Attributes to Entities and Entity Sets with Other Relationship Types
Double Ellipse		Multi-Valued Attributes
Double Rectangle		Weak Entity

Cardinality



Zero or one



Many



One



One (and only one)

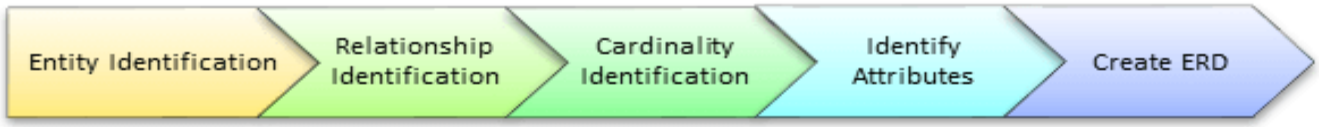


Zero or many



One or many

Steps to create an Entity Relationship Diagram (ERD)



Steps to Create an ER Diagram

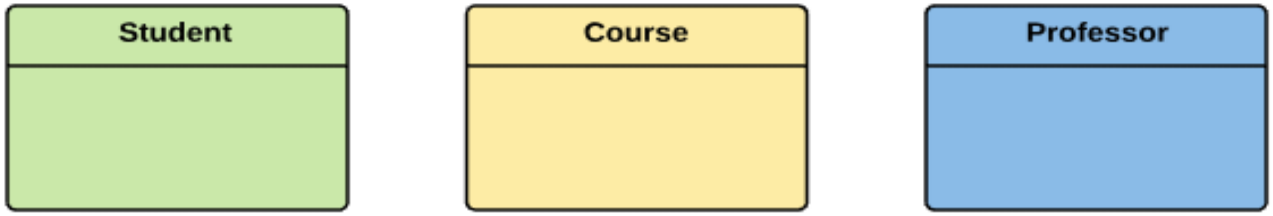
Let's study them with an Entity Relationship Diagram Example:

```
In a university, a Student enrolls in Courses. A student must be assigned to at
least one or more Courses. Each course is taught by a single Professor. To maint
ain instruction quality, a Professor can deliver only one course
```

Step 1) Entity Identification

We have three entities

- Student
- Course
- Professor

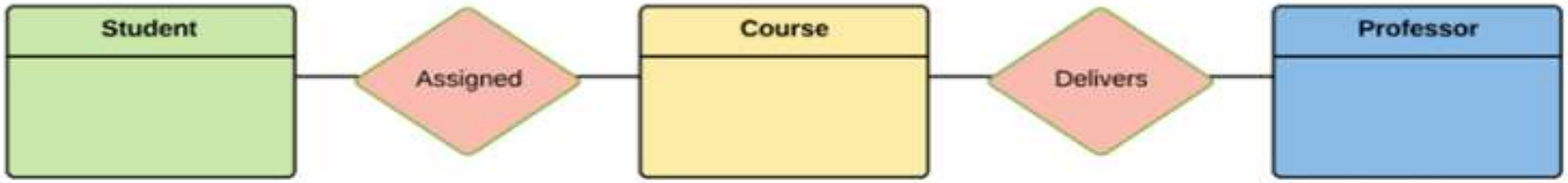


Steps to create an Entity Relationship Diagram (ERD)

Step 2) Relationship Identification

We have the following two relationships

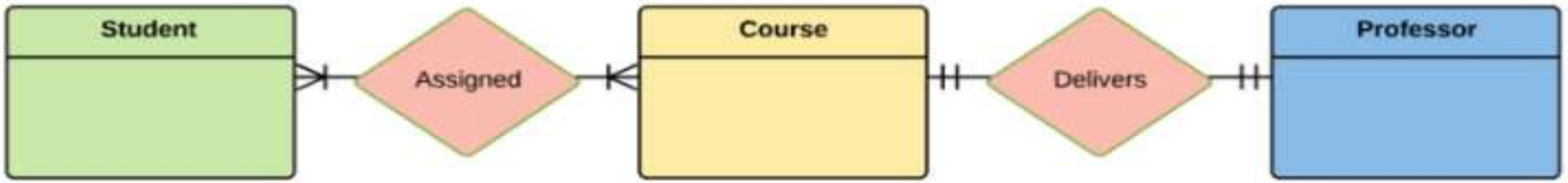
- The student is **assigned** a course
- Professor **delivers** a course



Step 3) Cardinality Identification

For them problem statement we know that,

- A student can be assigned **multiple** courses
- A Professor can deliver only **one** course



Steps to create an Entity Relationship Diagram (ERD)

Step 4) Identify Attributes

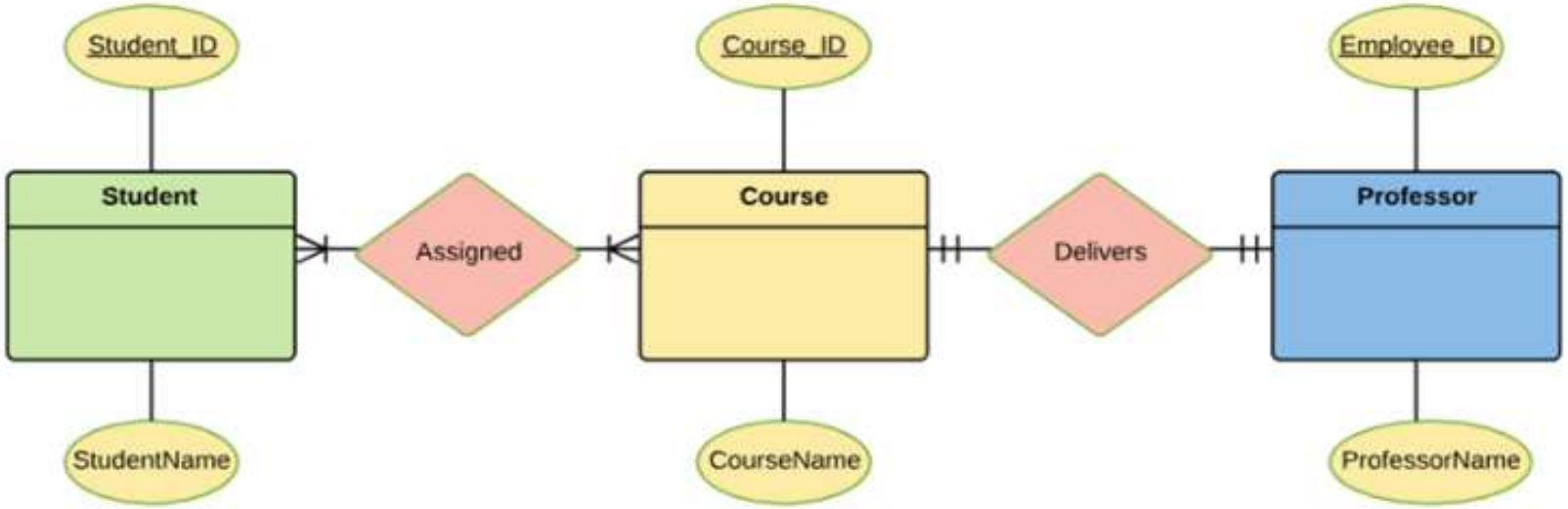
You need to study the files, forms, reports, data currently maintained by the organization to identify attributes. You can also conduct interviews with various stakeholders to identify entities. Initially, it's important to identify the attributes without mapping them to a particular entity.

Once, you have a list of Attributes, you need to map them to the identified entities. Ensure an attribute is to be paired with exactly one entity. If you think an attribute should belong to more than one entity, use a modifier to make it unique.

Once the mapping is done, identify the primary Keys. If a unique key is not readily available, create one.

Entity	Primary Key	Attribute
Student	Student_ID	StudentName
Professor	Employee_ID	ProfessorName
Course	Course_ID	CourseName

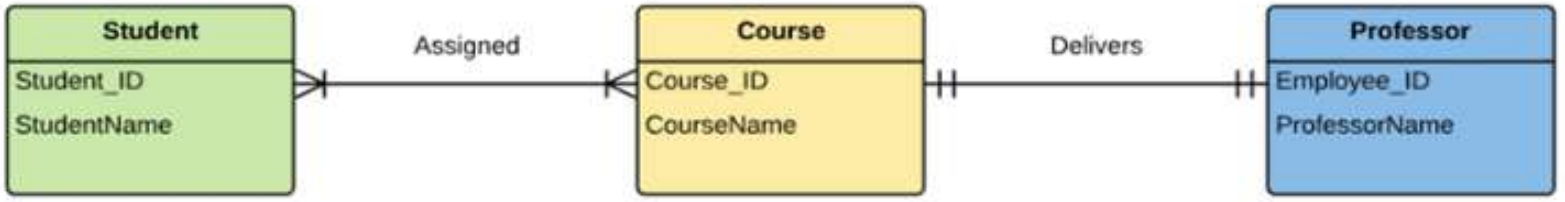
Steps to create an Entity Relationship Diagram (ERD)



For Course Entity, attributes could be Duration, Credits, Assignments, etc. For the sake of ease we have considered just one attribute.

Step 5) Create the ERD Diagram

A more modern representation of Entity Relationship Diagram Example



Best Practices for Developing Effective ER Diagrams

Here are some best practice or example for Developing Effective ER Diagrams.

- Eliminate any redundant entities or relationships
- You need to make sure that all your entities and relationships are properly labeled
- There may be various valid approaches to an ER diagram. You need to make sure that the ER diagram supports all the data you need to store
- You should assure that each entity only appears a single time in the ER diagram
- Name every relationship, entity, and attribute are represented on your diagram
- Never connect relationships to each other
- You should use colors to highlight important portions of the ER diagram

DBMS Keys

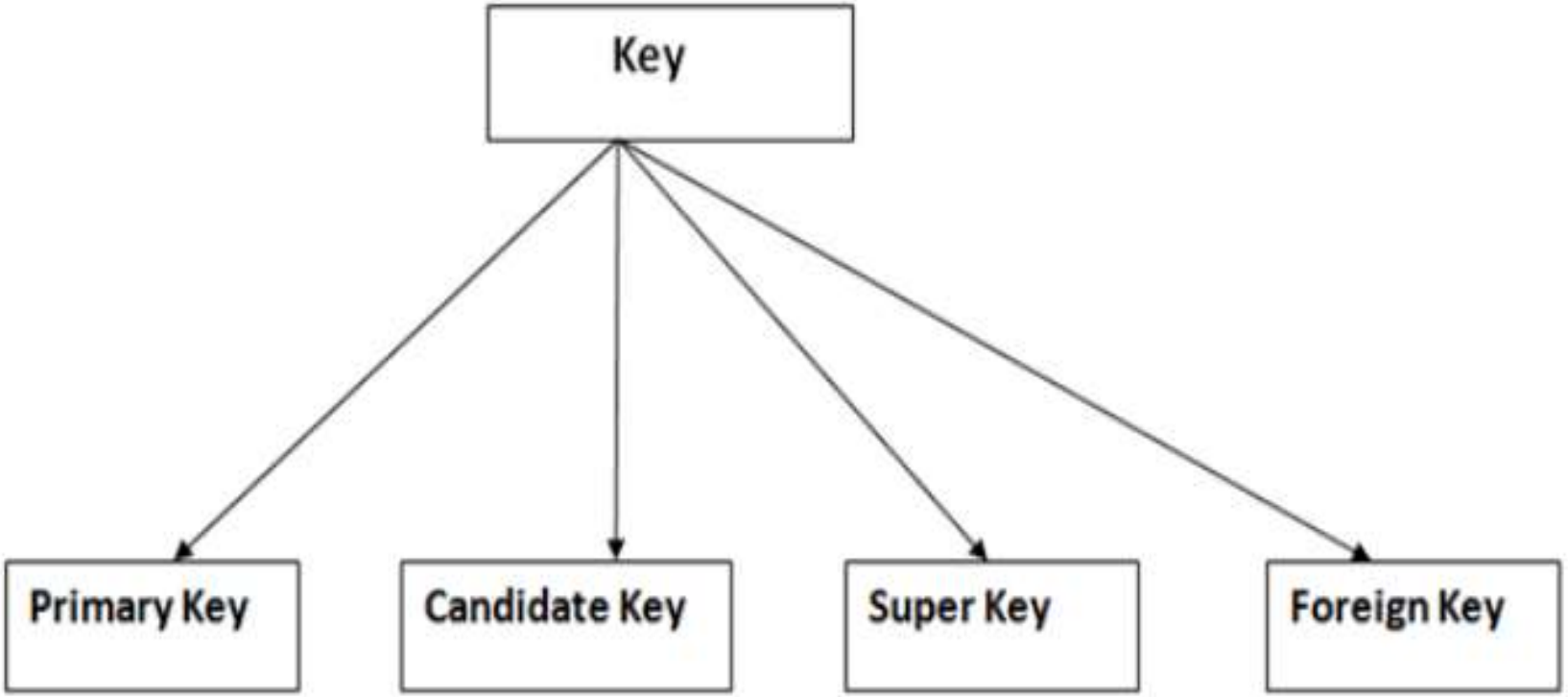
- Keys play an important role in the relational database.
- It is used to uniquely identify any record/tuple/row of data from the table. It is also used to establish and identify relationships between tables.

For example: In Student table, ID is used as a key because it is unique for each student. In PERSON table, passport_number, license_number, SSN are keys since they are unique for each person.





Types of key



Super Key

4.5.1 Super Key

A super key is a set of one or more attributes that, taken collectively, allow us to identify uniquely an entity in the entity set. For example Sid attribute in the student relation shows unique property because every student having unique student id or enrollment no. So Sid is a super key and we can create more super key with the use of Sid or the combination with Sid. Name is not a super key because several person might have the same name.

Example of superkey from Student Relation

(*i*) {Sid}
(*ii*) {Sid + Name}
(*iii*) {Sid + Name + Address}
(*iv*) {Sid + Name + Address + Ph no.}
(*v*) {Sid + Name + Address + Ph no. + College}

Sid	Name	Address	Ph no.	College
101	Ravi	Indore	25432	T.I.T
102	Rajesh	Indore	25499	T.I.T
103	Animesh	Bhopal	—	OIST
104	Ganesh	Shahdol (a)	26289	LNCT
105	Rahul	Guna	26239	OIST
106	Pankaj	Anuppur	26139	LNCT
107	Rahul	Bhopal	26425	BIST

Fig. 4.16 *Student Relation.*

Candidate Key

101	Ravi	Indore	25432	T.I.T
102	Rajesh	Indore	25499	T.I.T
103	Animesh	Bhopal	—	OIST
104	Ganesh	Shahdol (a)	26289	LNCT
105	Rahul	Guna	26239	OIST
106	Pankaj	Anuppur	26139	LNCT
107	Rahul	Bhopal	26425	BIST

Fig. 4.16 Student Relation.

4.5.2 Candidate Key

A candidate key is a field or combination of fields that can uniquely identify each record in that table Minimal super key is called candidate key. Any candidate key can be a primary key, but it depends upon the requirement. It means it is a super key such that subset will not be key or superkey. For example, Sid is a candidate key and name + Address is also a candidate key because it is sufficient to distinguish among members of student entity. Although the attribute Sid + Name together can distinguish student entity but there combination does not form a candidate key, since the attribute Sid alone is a candidate key. Example of candidate key (*i*) {Sid} (*ii*) {Name, Address}, (*iii*) {Phno, College}. If we use any attribute as a candidate key then we can not use the attribute again in any other combination for candiate key.

Primary Key

4.5.3 Primary key

A primary key is a column or attribiute in the table (relation) whose purpose is to uniquely identify records in the table. Primary key → UNIQUE + NOT NULL. The primary key should be selected in the manner such that it is unique and not null, means every row having unique values but not null. For example, phno is unique for every student in student table but it is not a primary key because the Row no. 3, having Sid "103" and Name "Animesh" not have phno, means this row value is null. The primary key is chosen from the candidate key. So Sid is the primary key. Because it uniquely identify a student from the student entity and Sid can never be null. It must have some value.

e.g. — Sid in the student table

Foreign Key

4.5.5 Foreign key

A foreign key (FK) is a field or group of field in a database record that points to a key field or group of fields forming a key of another database record in some other table. Usually a foreign key in one table refers to the primary key (PK) of another table. This way we can link two tables (relations) for retrieving the data jointly.

Consider an example which is shown in fig. 4.17.

In First table (Student) Sid is a primary key. In Marks table Sid is a foreign key for relating two tables or perform join operation on those two tables.

A foreign key value must match an existing value in the parent table or be Null. Foreign keys are based on data values and are purely logical, not physical. If we want to insert 104 value in Sid column then we can insert it. But if we want to insert 90 as the value of Sid in marks table, then we cannot insert it because it is not present in the base table (student).

↓ Primary key (PK)

Sid	Name	Address
101		
102		
103		
104		

(a) Student entity

Foreign key (FK)

Cid	Total	Sid
		101
		101
		102
		102
		104
		102

(b) Marks entity

Fig. 4.17 *Student and marks entity*

It means we only insert those value which are present in base table. If we delete the base table, automatically primary key and foreign key relationship is broken.

Foreign Key

Employee Table		
Emp_Id	Emp_name	Did
1	Sachin	20
2	Suhas	10
3	Jay	20
4	Om	10

Department Table	
Did	Dept_name
10	HR
20	TIS
30	L&D

- In the above example Employee table has Did as foreign key reference to Did column in Department table this is called as referential integrity.
- Here we are forcing the database to check the value of Did column from the department table while inserting any value in Employee table. This helps to maintain data consistency.

FOREIGN KEY

- A FOREIGN KEY is a key used to link two tables together.

- A FOREIGN KEY is a field (or collection of fields) in one table that refers to the PRIMARY KEY in another table.

- The table containing the foreign key is called the child table, and the table containing the primary key is called the parent table.

Look at the following two tables:

"Persons" table:

PersonID	LastName	FirstName	Age
1	Hansen	Ola	30
2	Svendson	Tove	23
3	Pettersen	Kari	20

"Orders" table:

OrderID	OrderNumber	PersonID
1	77895	3
2	44678	3
3	22456	2
4	24562	1

•Notice that the "PersonID" column in the "Orders" table points to the "PersonID" column in the "Persons" table.

•The "PersonID" column in the "Persons" table is the PRIMARY KEY in the "Persons" table.

•The "PersonID" column in the "Orders" table is a FOREIGN KEY in the "Orders" table.

•The FOREIGN KEY constraint also prevents invalid data from being inserted into the foreign key column, because it has to be one of the values contained in the table it points to.

Example ER Diagram

Example 11— Construct an E-R diagram for hospital with a set of patient and medical doctors.

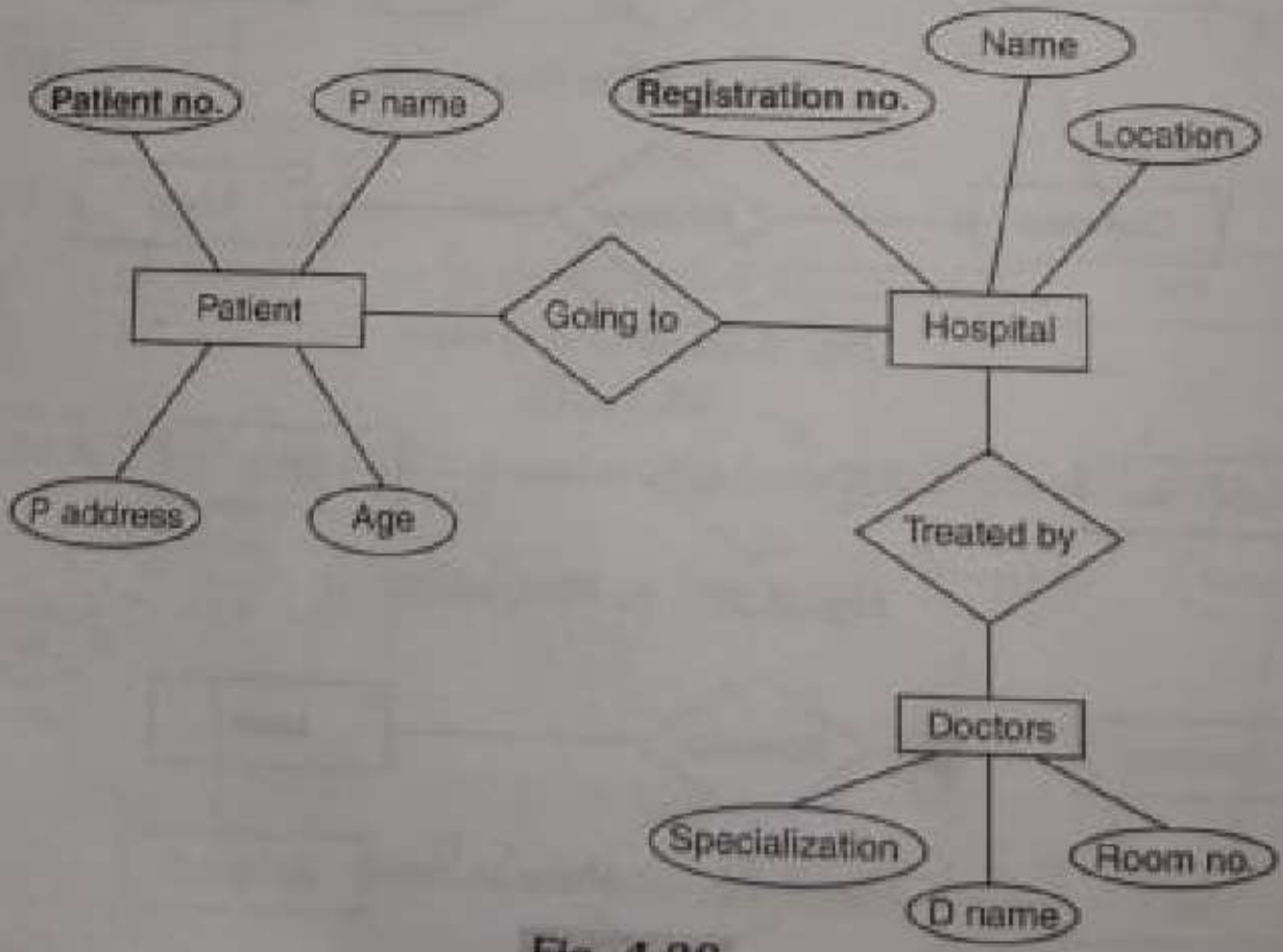
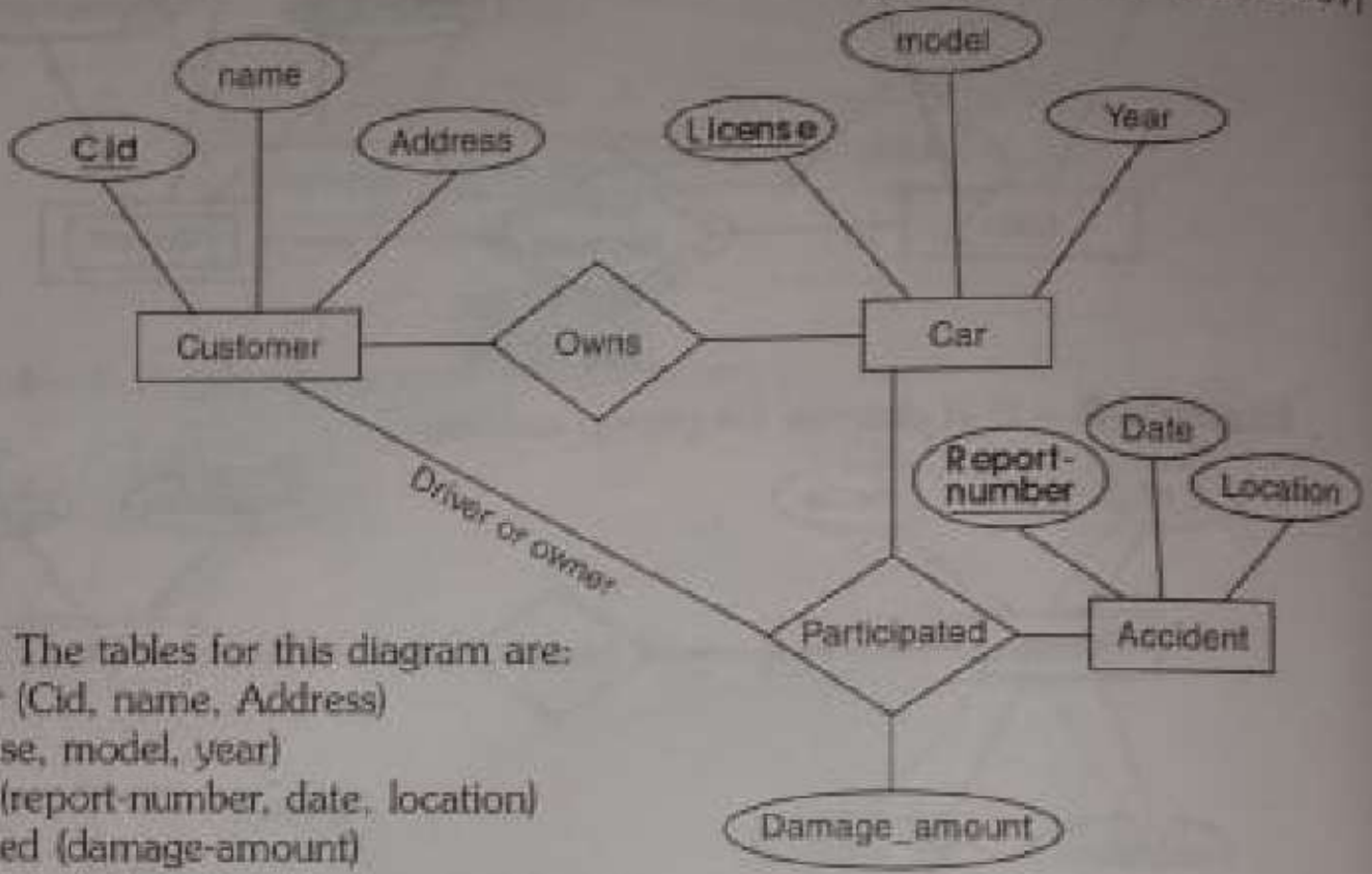


Fig. 4.28

Example ER Diagram

Example 10— Construct an E-R diagram of a car insurance company that has a set of customers each of whom owns one or more cars. Each car has associated with it 0 to any number of recorded accidents Construct appropriate tables for the same.

[R.G.P.V June 2006, 2007]



Solution The tables for this diagram are:
Customer (Cid, name, Address)
Car (license, model, year)
Accident (report-number, date, location)
Participated (damage-amount)

Fig. 4.27

Example ER Diagram

Example 3.8.3 : Construct an ER diagram for a hospital with a set of patients and the set of medical doctors associated with each patient a record of various text and examination conducted.

(10 Marks)

Solution :

(1) Identify Entities

(i) Hospital (ii) Patient

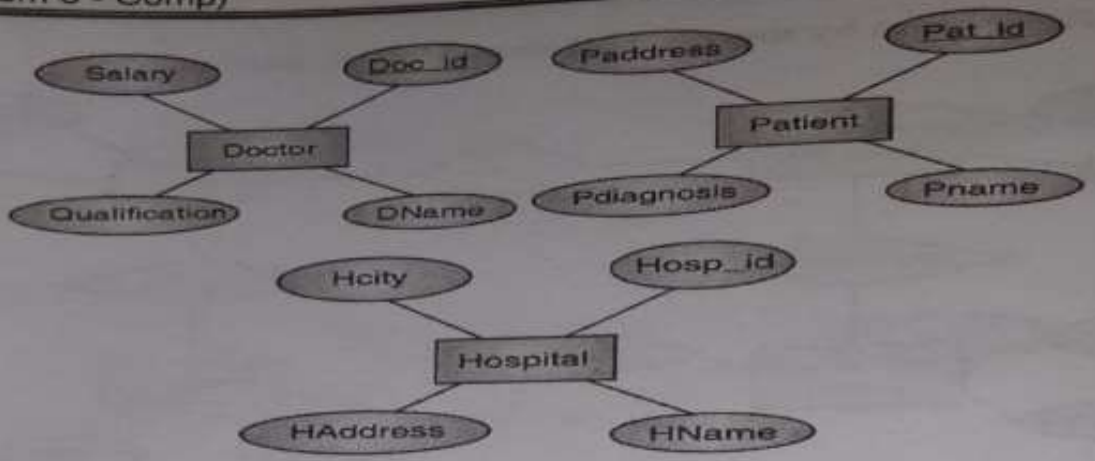
(iii) Doctors (iv) Medical-record (Record of various test and examination conducted)

(2) Identify Attributes

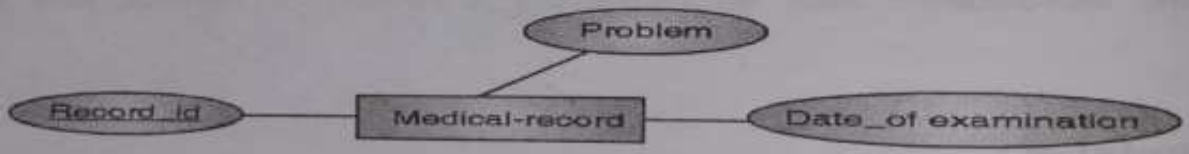
(i) Hospital (Hosp_id, HName, HAddress, Hcity)

(ii) Patient (Pat_id, Pname, Pdiagnosis, Padress)

(iii) Doctor (Doc_id, DName, Qualification, salary)

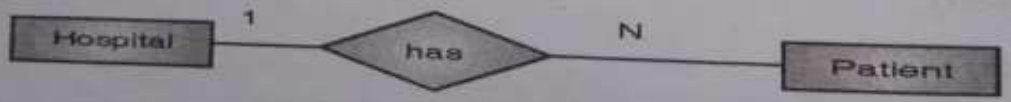


(iv) Medical_Record (Record_id, Date_of examination, Problem)

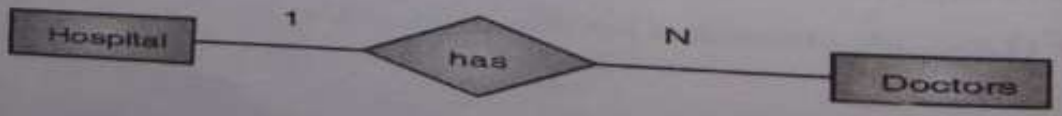


Identify relationships

(a) Hospital has a set of patients



(b) Hospital has a set of doctors



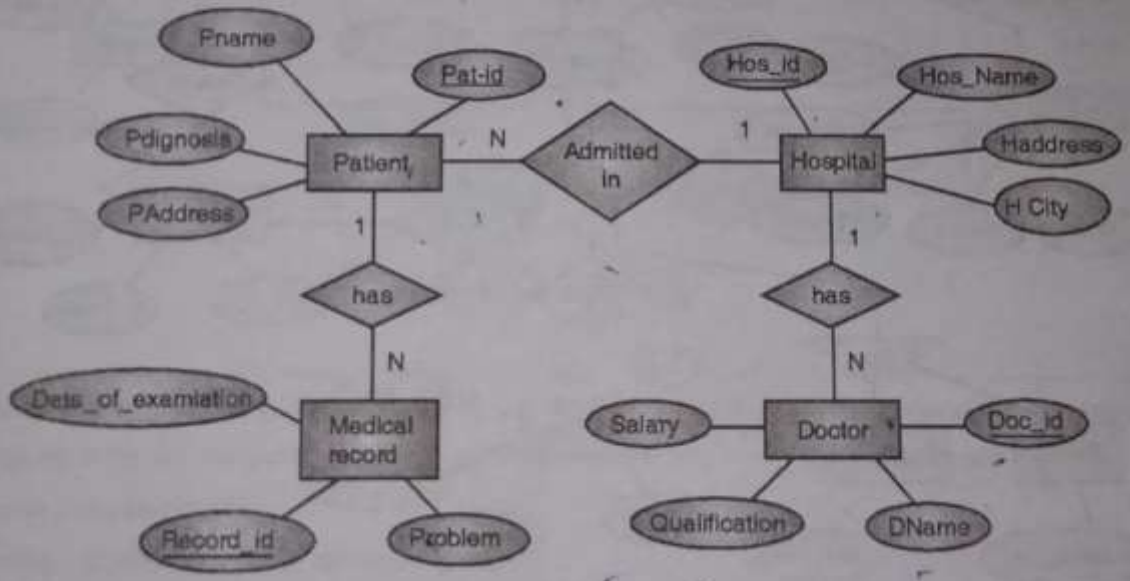
(c) Doctors are associated with each patient



(d) Each patient has record of various test and examination conducted



(4) Construct ER Model from merging all above relationship



Example ER Diagram

Example 3.8.2 : Construct on E-R diagram for a car-insurance company that has a set of customers each of whom owns one or more cars. Each car has associated with it zero to any number of recorded accidents. **(10 Marks)**

Solution :

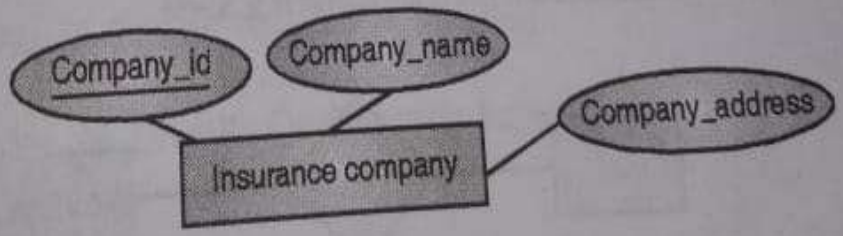
(1) Identify all entities

(a) Insurance company (b) Customer

(c) Car (d) Accidents

(2) Identify all attributes

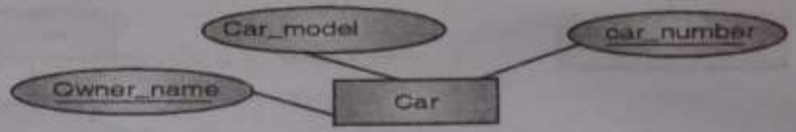
(a) Company entity



(b) Customer entity



(c) Car entity

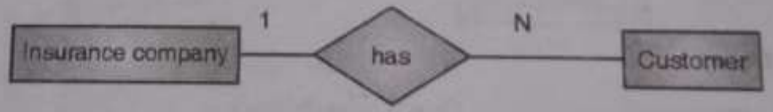


(d) Accidents



(3) Identify all relationship

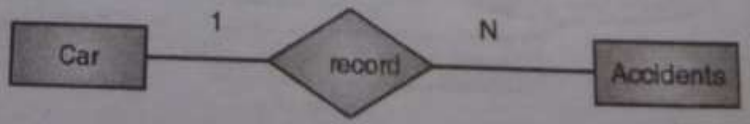
a) Car insurance company has a set of customers



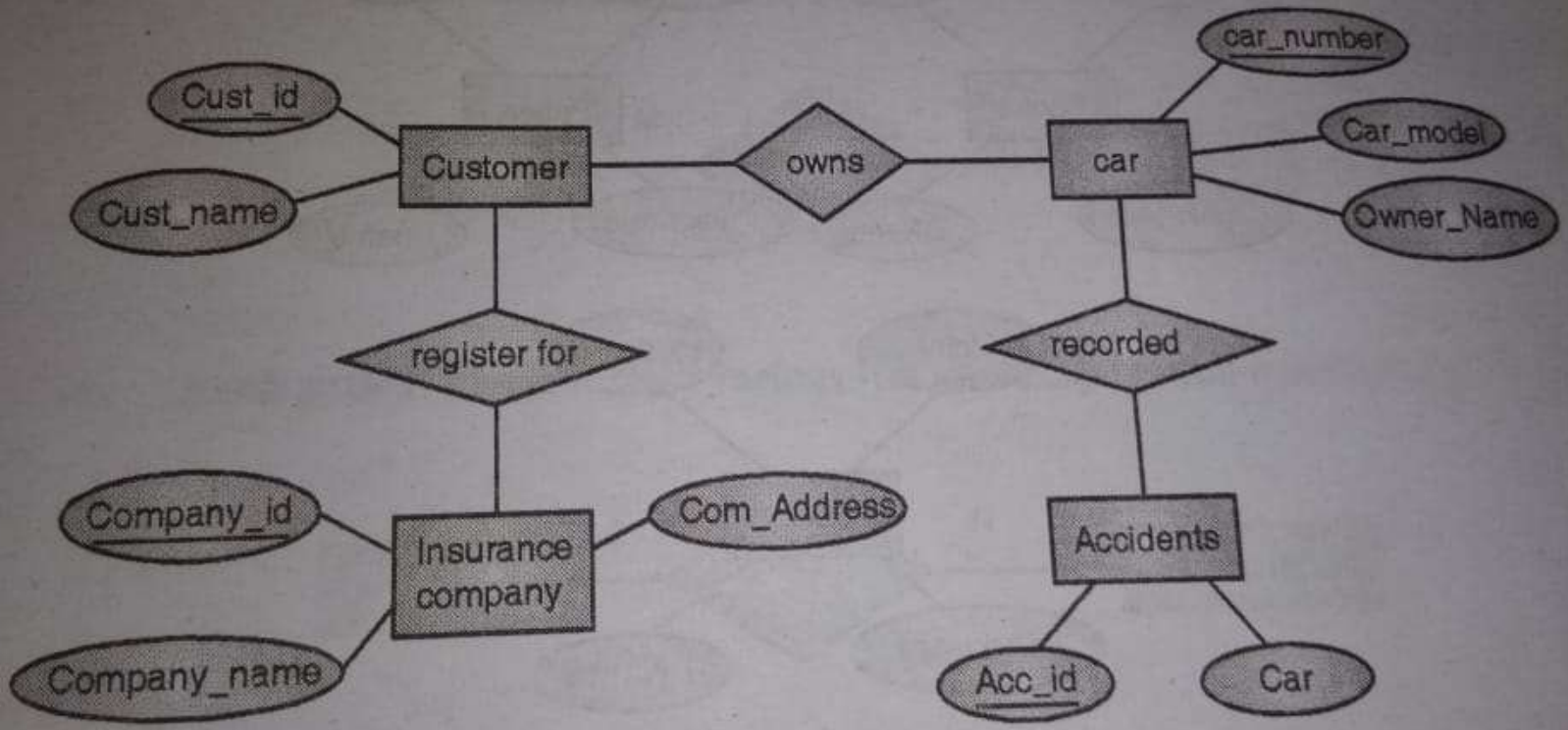
b) Customer owns one or more car



c) Each car associated with zero or any number of accidents



(4) Construct ER diagram by merging all above relationships



Example 12— Construct an E–R diagram for a hospital with a set of patient and a set of medical doctors. Associate with each patient a log of various tests and Examination Conducted. [*R.G.P.V June 2004*]

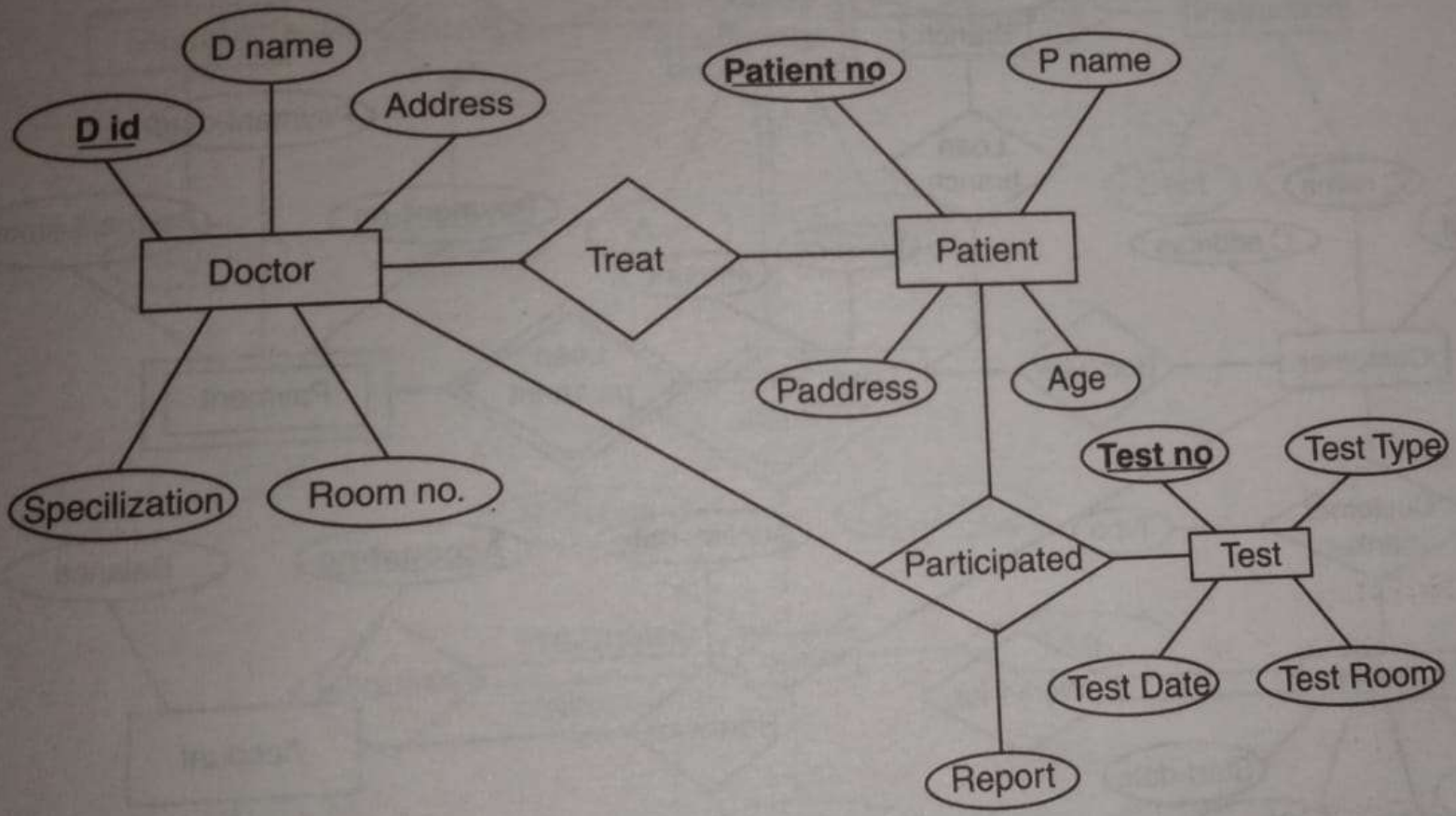


Fig. 4.29

Example 19— Construct an E–R diagram for online book store.

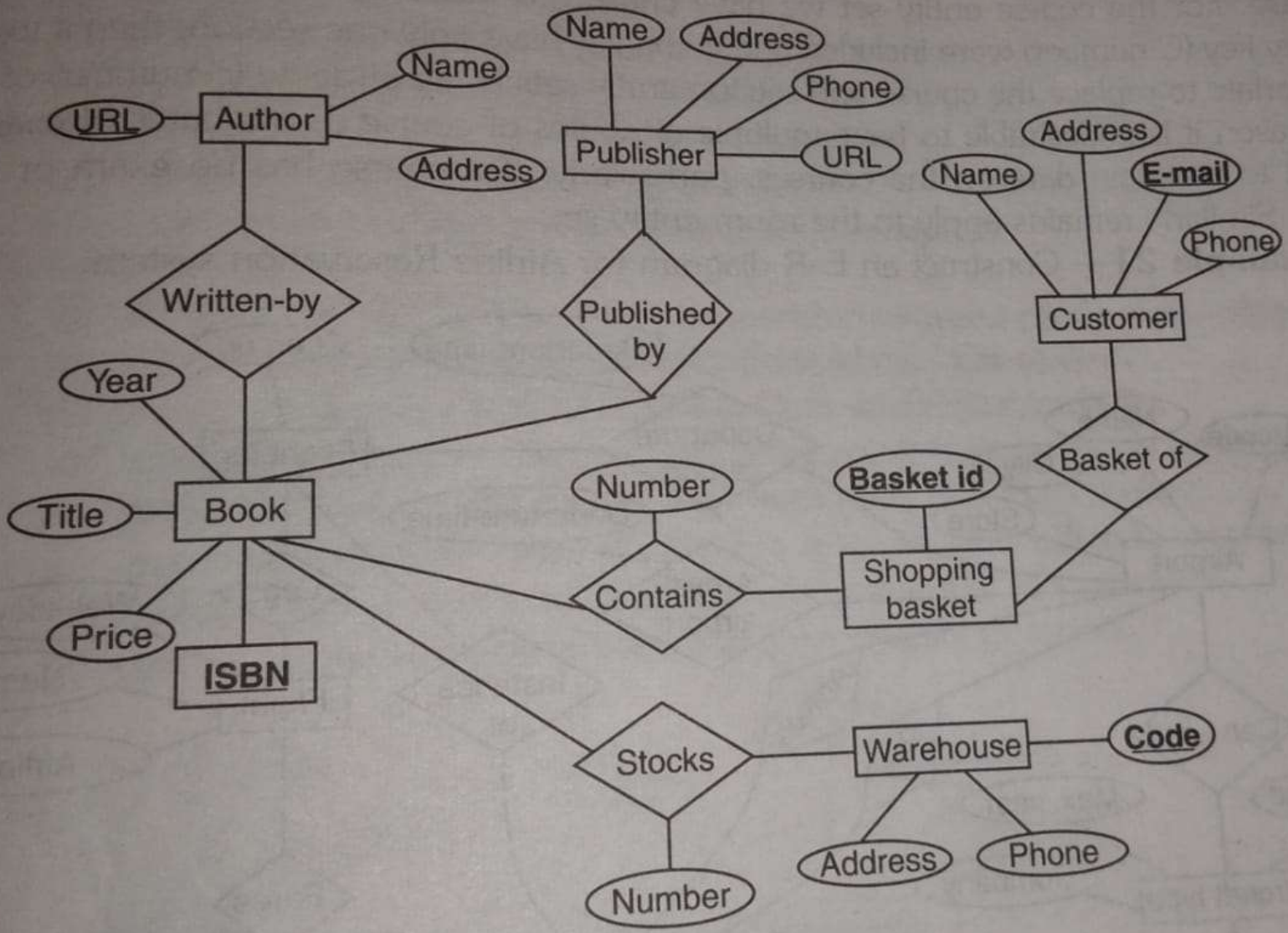


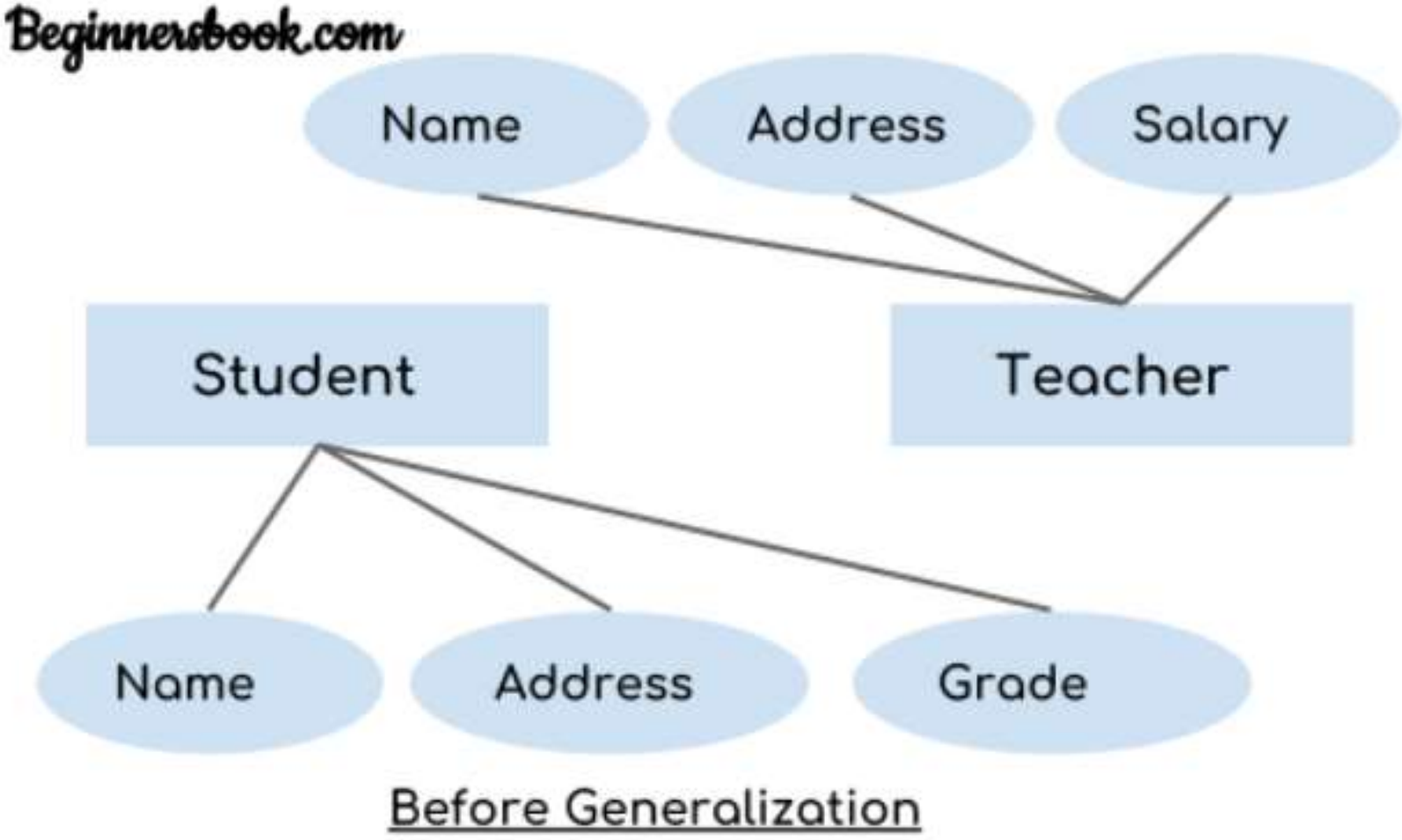
Fig. 4.36

Enhanced/Extended Entity-Relationship (EER) model

- DBMS Generalization
  - Generalization is a process in which the common attributes of more than one entities form a new entity. This newly formed entity is called generalized entity.
  - Generalization is more like subclass and superclass system.
  - Generalization uses the bottom-up approach.

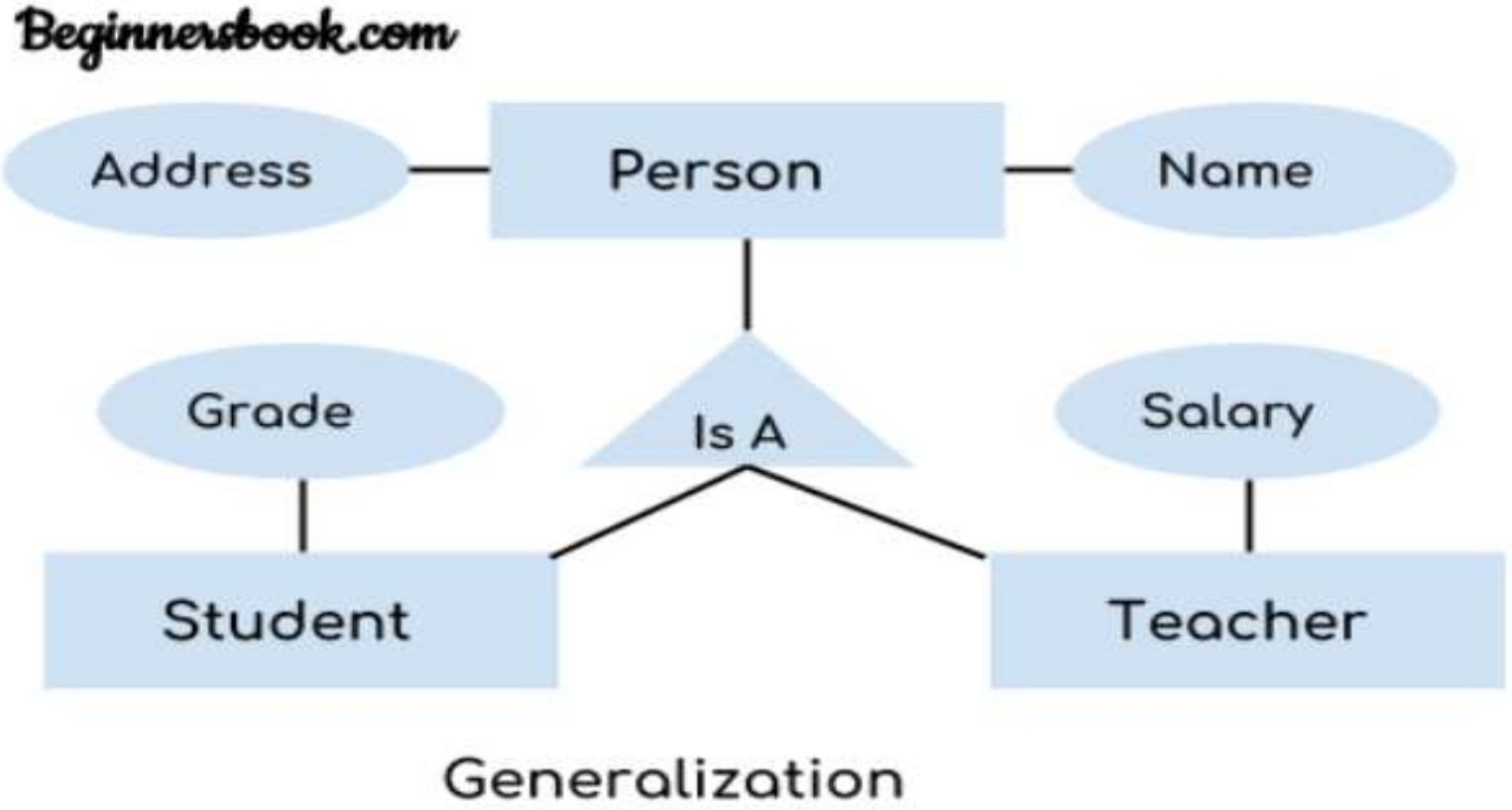
Enhanced/Extended Entity-Relationship (EER) model

- DBMS Generalization example



Enhanced/Extended Entity-Relationship (EER) model

- DBMS Generalization example

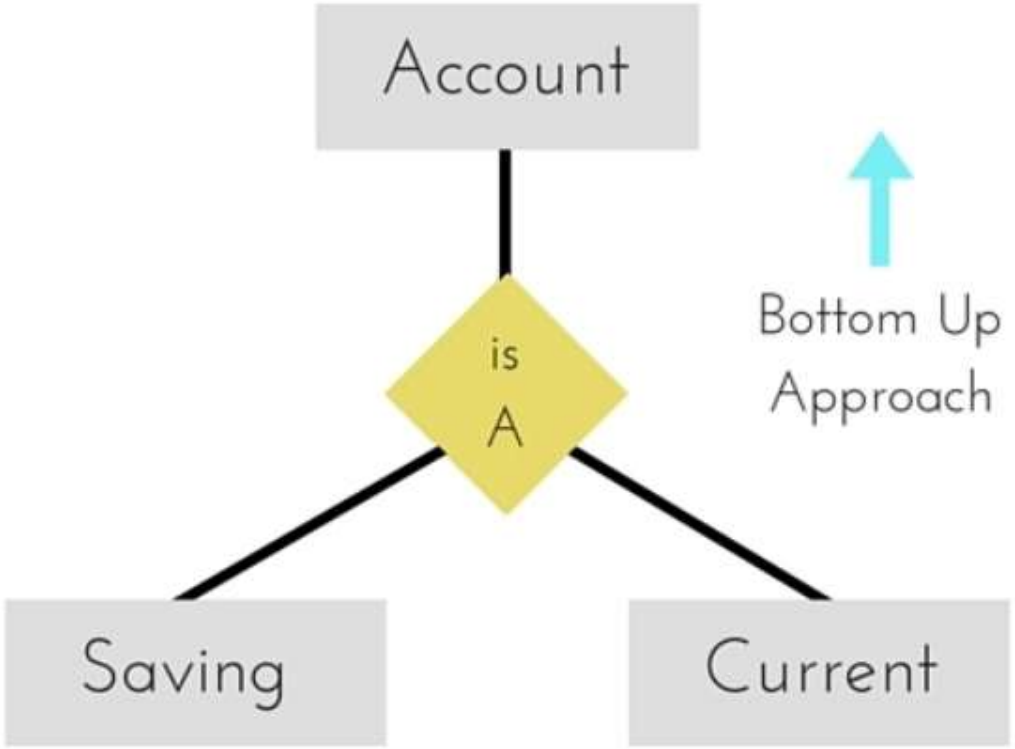


Generalization

The new generalized entity can further combine together with lower level entity to create a further higher level generalized entity.

Enhanced/Extended Entity-Relationship (EER) model

- DBMS Generalization example



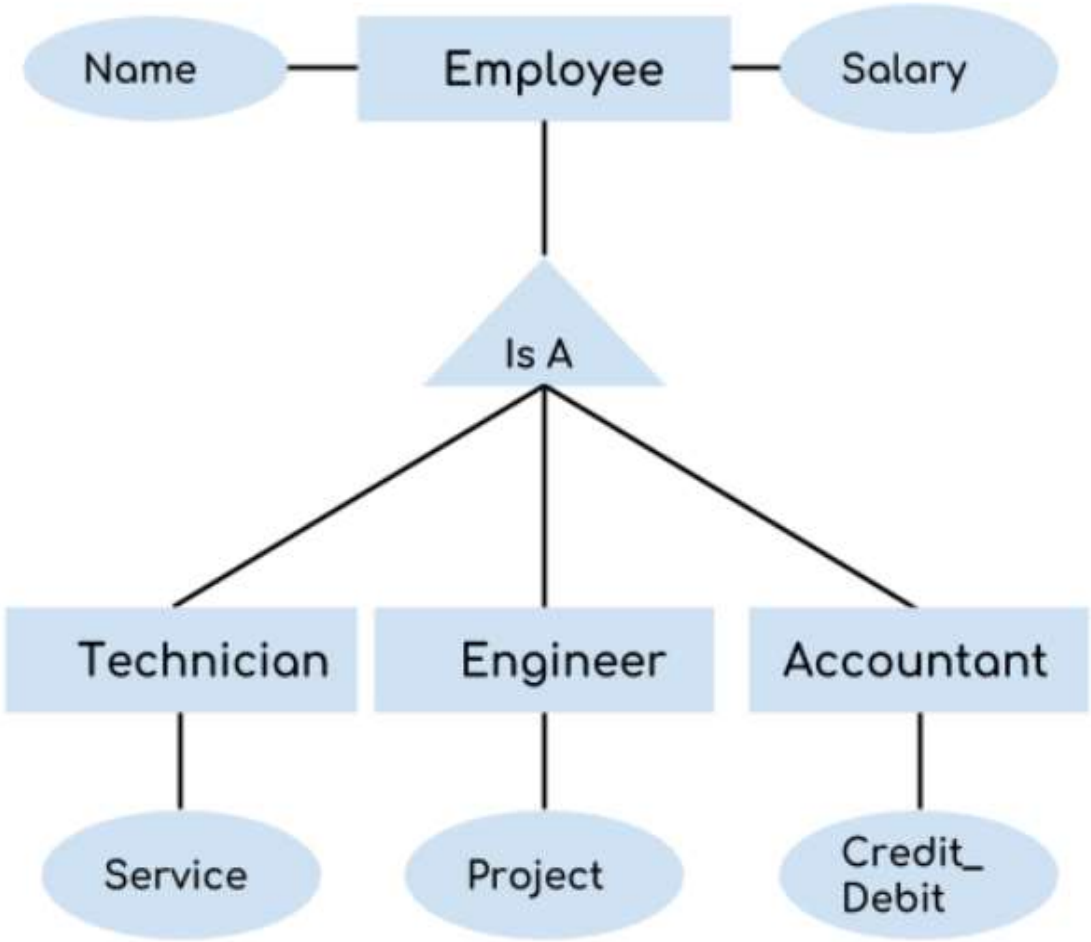
For example, **Saving** and **Current** account types entities can be generalised and an entity with name **Account** can be created, which covers both.

Enhanced/Extended Entity-Relationship (EER) model

- DBMS Specialization
  - Specialization is a process in which an entity is divided into sub-entities. It is a reverse process of generalization, in generalization two entities combine together to form a new higher level entity. Specialization is a top-down process.
  - The idea behind Specialization is to find the subsets of entities that have few distinguish attributes.
  - Normally, the superclass is defined first, the subclass and its related attributes are defined next, and relationship set are then added.

Enhanced/Extended Entity-Relationship (EER) model

- DBMS Specialization example

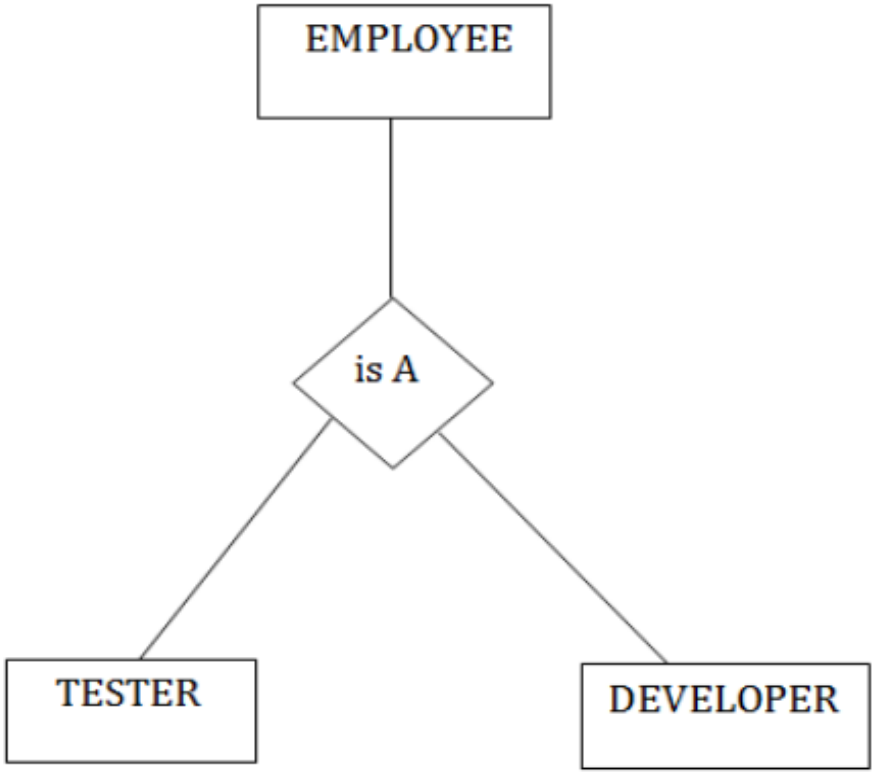


Specialization

Enhanced/Extended Entity-Relationship (EER) model

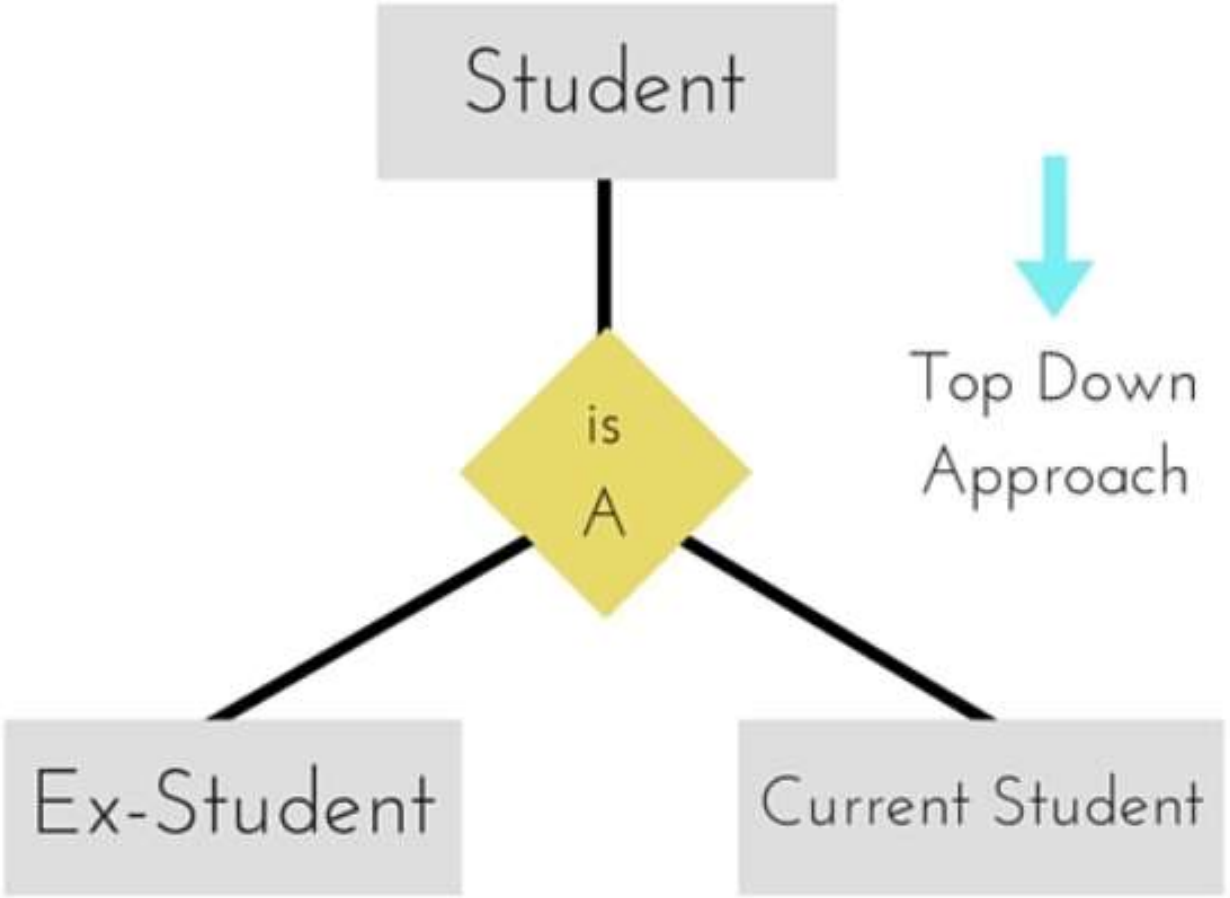
- DBMS Specialization example

For example: In an Employee management system, EMPLOYEE entity can be specialized as TESTER or DEVELOPER based on what role they play in the company.



Enhanced/Extended Entity-Relationship (EER) model

- DBMS Specialization example



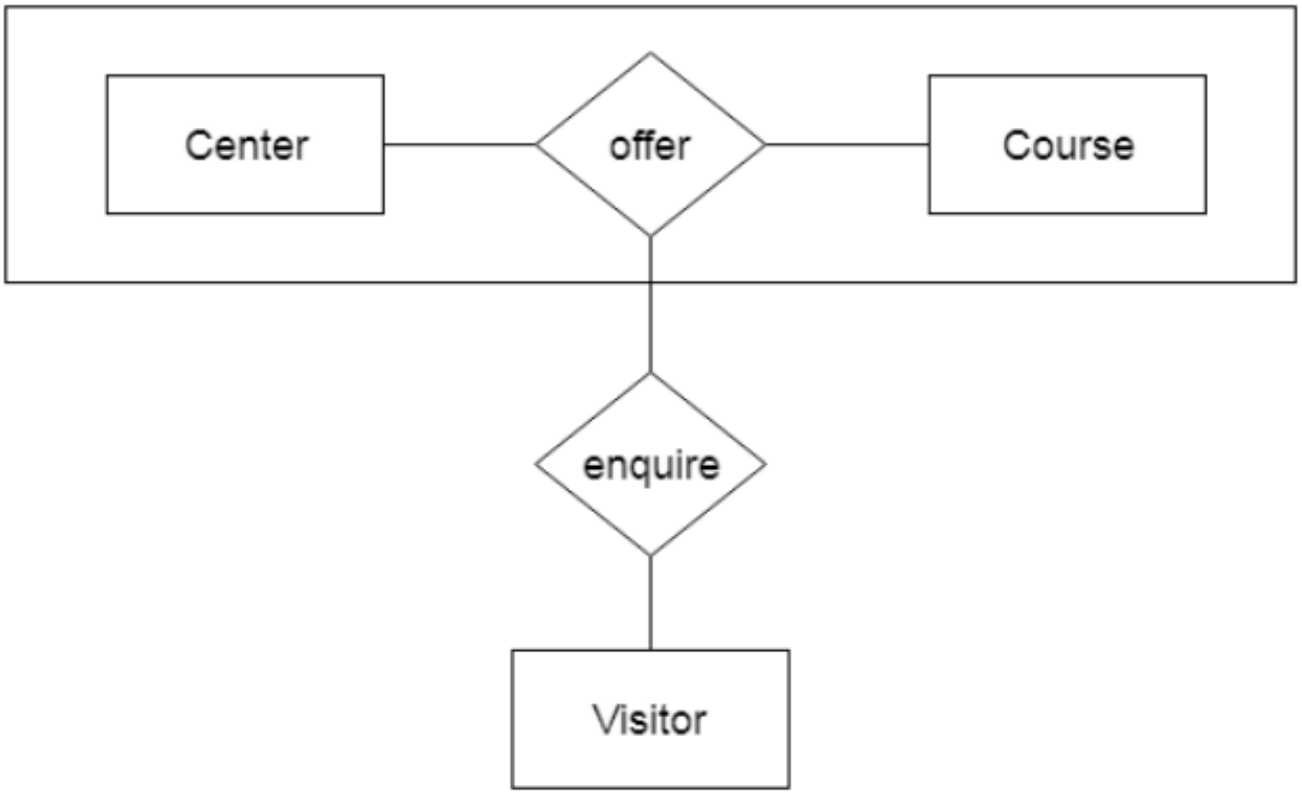
Enhanced/Extended Entity-Relationship (EER) model

- DBMS Aggregration
  - Aggregation is a process in which a single entity alone is not able to make sense in a relationship so the relationship of two entities acts as one entity.
  - In aggregation, relationship with its corresponding entities is aggregated into a higher level entity.

Enhanced/Extended Entity-Relationship (EER) model

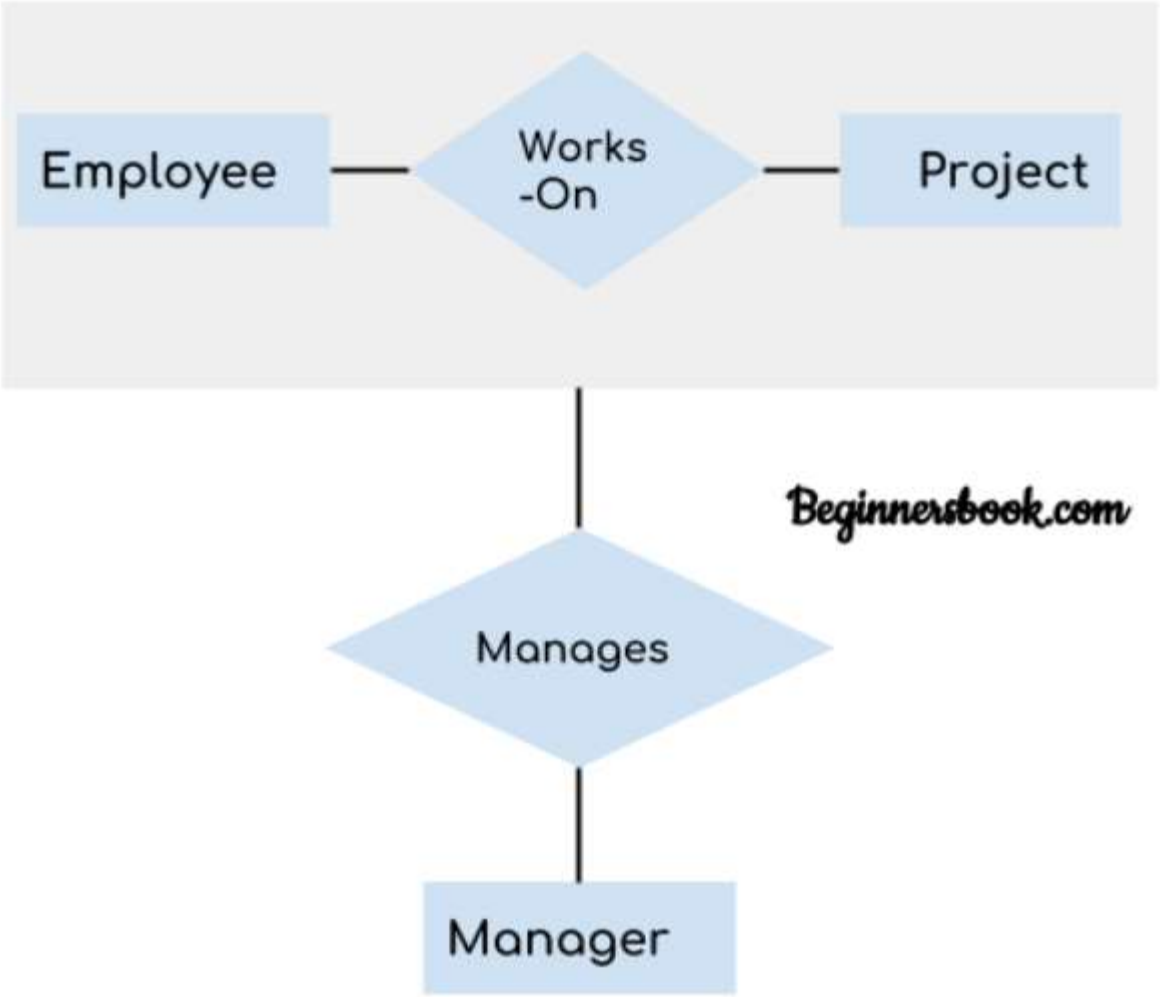
- DBMS Aggregration example

For example: Center entity offers the Course entity act as a single entity in the relationship which is in a relationship with another entity visitor. In the real world, if a visitor visits a coaching center then he will never enquiry about the Course only or just about the Center instead he will ask the enquiry about both.



Enhanced/Extended Entity-Relationship (EER) model

- DBMS Aggregration example



Enhanced/Extended Entity-Relationship (EER) model

- DBMS Aggregration example explained

➢ In real world, we know that a manager not only manages the employee working under them but he has to manage the project as well. In such scenario if entity “Manager” makes a “manages” relationship with either “Employee” or “Project” entity alone then it will not make any sense because he has to manage both. In these cases the relationship of two entities acts as one entity. In our example, the relationship “Works-On” between “Employee” & “Project” acts as one entity that has a relationship “Manages” with the entity “Manager”.

Enhanced/Extended Entity-Relationship (EER) model

- DBMS Aggregation example

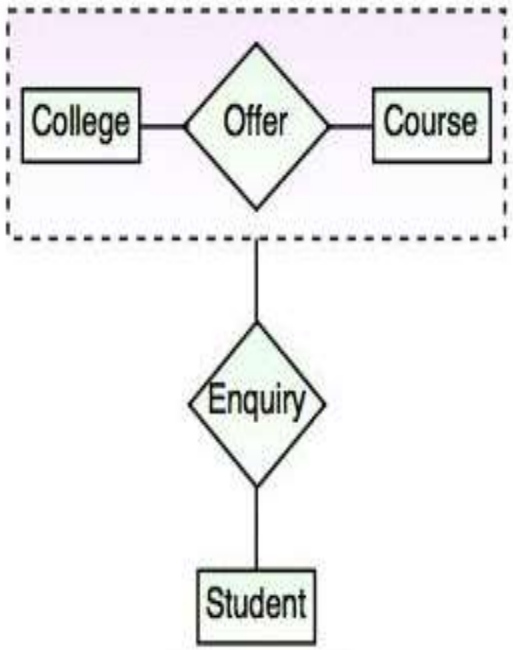


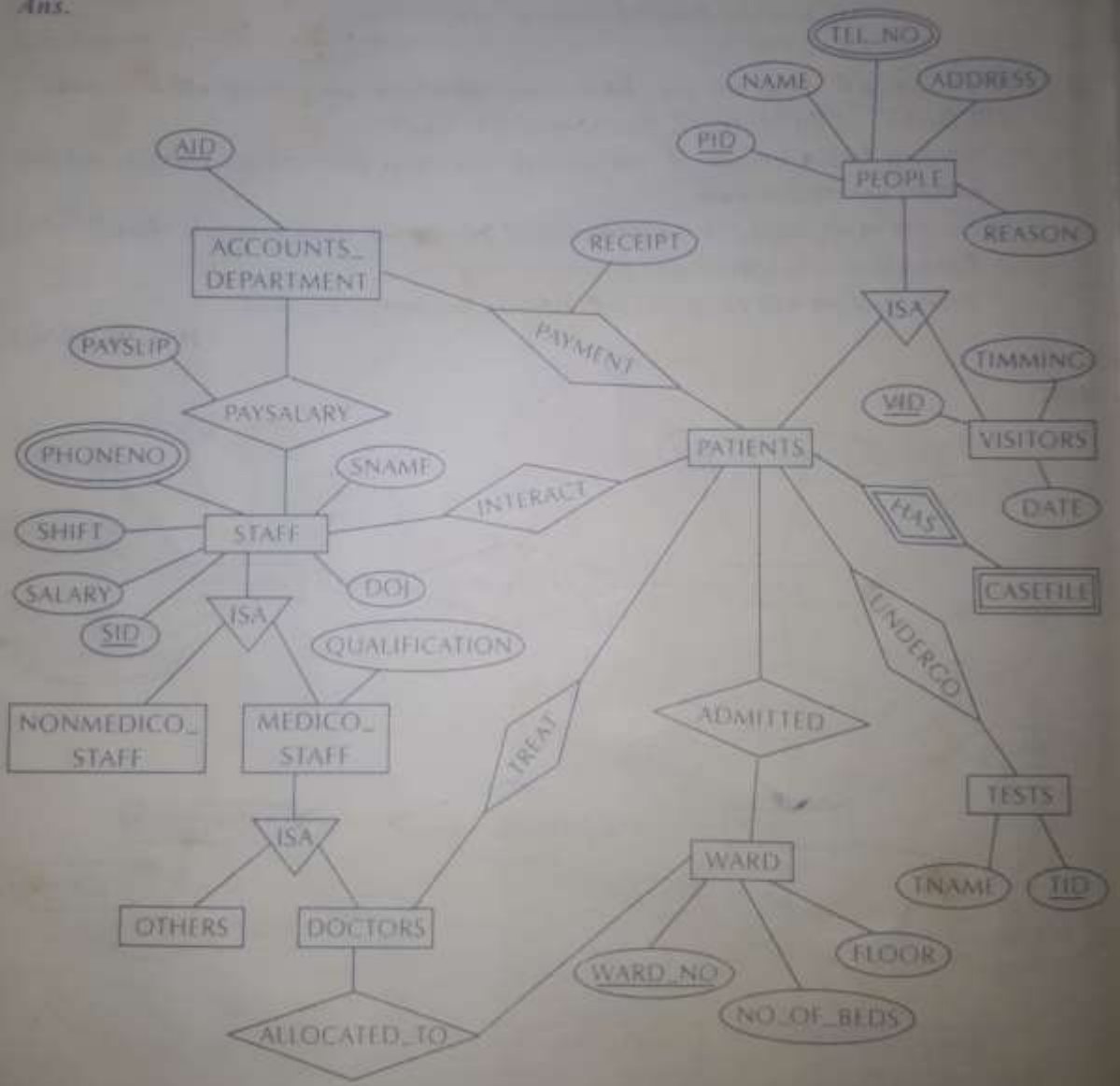
Fig. Aggregation

In the above example, the relation between College and Course is acting as an Entity in Relation with Student.

Q.3. ***Construct an ER diagram for Hospital with a set of patients and a set of medical doctors. Associate with each patient a log of various tests and examinations conducted. Write relation schema for the ER design.***

[Dec. 05 (10 M)]

Ans.



Q.2. *Draw an ER diagram for University Database consisting of 4 entities : STUDENT, DEPARTMENT, CLASS and FACULTY.*

- *Student has a unique id, the student can enrol for multiple classes and has atmost one major class.*
- *Faculty must belong to department and faculty can teach multiple class.*
- *Each class is taught by one faculty.*
- *Every student will get grade for the class he/she has enrolled.*

[Dec. 04 (10 M)]

Ans.

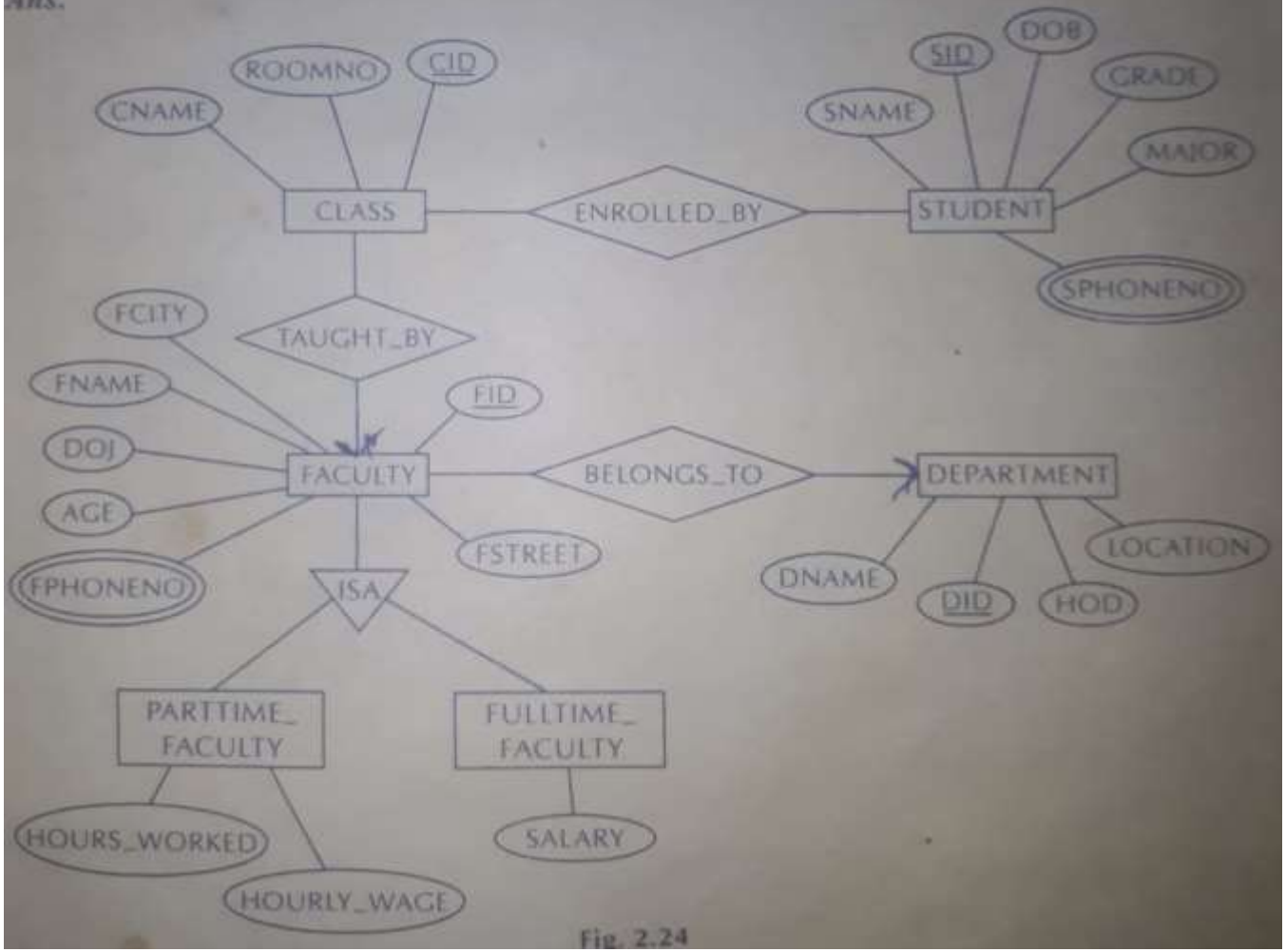
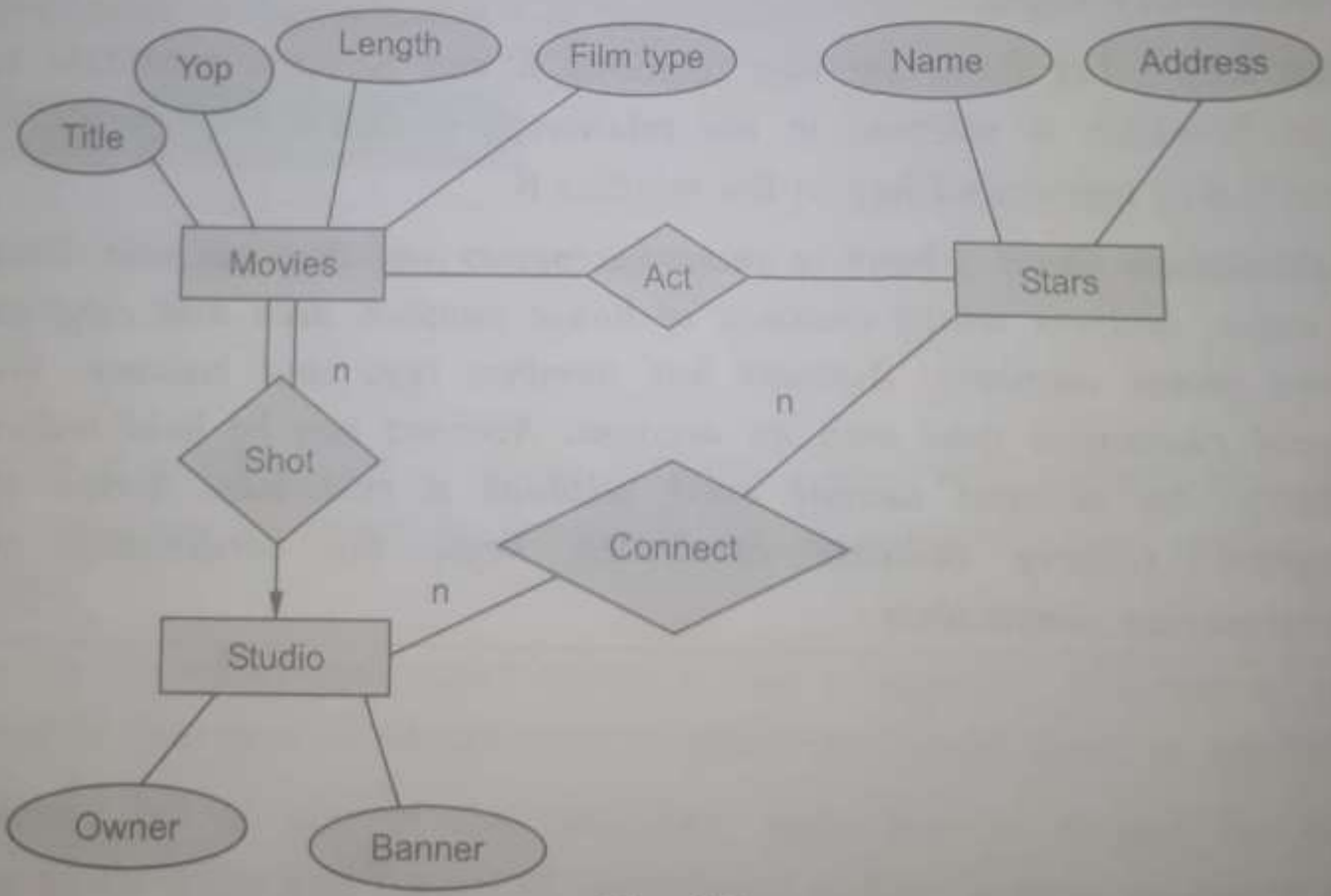


Fig. 2.24

Information about films contains information about movies, stars and studios. Movies have a title, year of production, length and the film type. Stars have a name and address. Studios have a owner and a banner. Movies are shot in studios which own them. A movie is shot in only one studio. Stars are connected to one or more studios but can act in any film which may or may not be owned by the studio. Arrive at an E-R diagram. Clearly indicate attributes, keys, the cardinality ratios and participation constraints. **Marks 8**



Q.8 *Draw E-R diagram for online railway ticket reservation system. Convert E-R diagram into tables.* **May-11, Marks 10**

Ans. : (See Fig. 2.13 on next page.)

Railway reservation database will have following tables :

1) Train (Train_No, Train_ Name, No_of_reserved_seats, No_of _vacant _seats)

2) Type_Info (Train_No, Type, Max _ Seats)

3) Route (Route_No, Source, Destination, Journey_hrs)

4) Passenger (Passenger _ Id, FName , MName, LName, Address, Ph_No1, Ph_No2, Age)

5) Reservation (Seat_No, Passenger_ID, Train_No, Route_No, Dept_Date, Dept_Time, Fare)

Note : Primary keys are underlined.

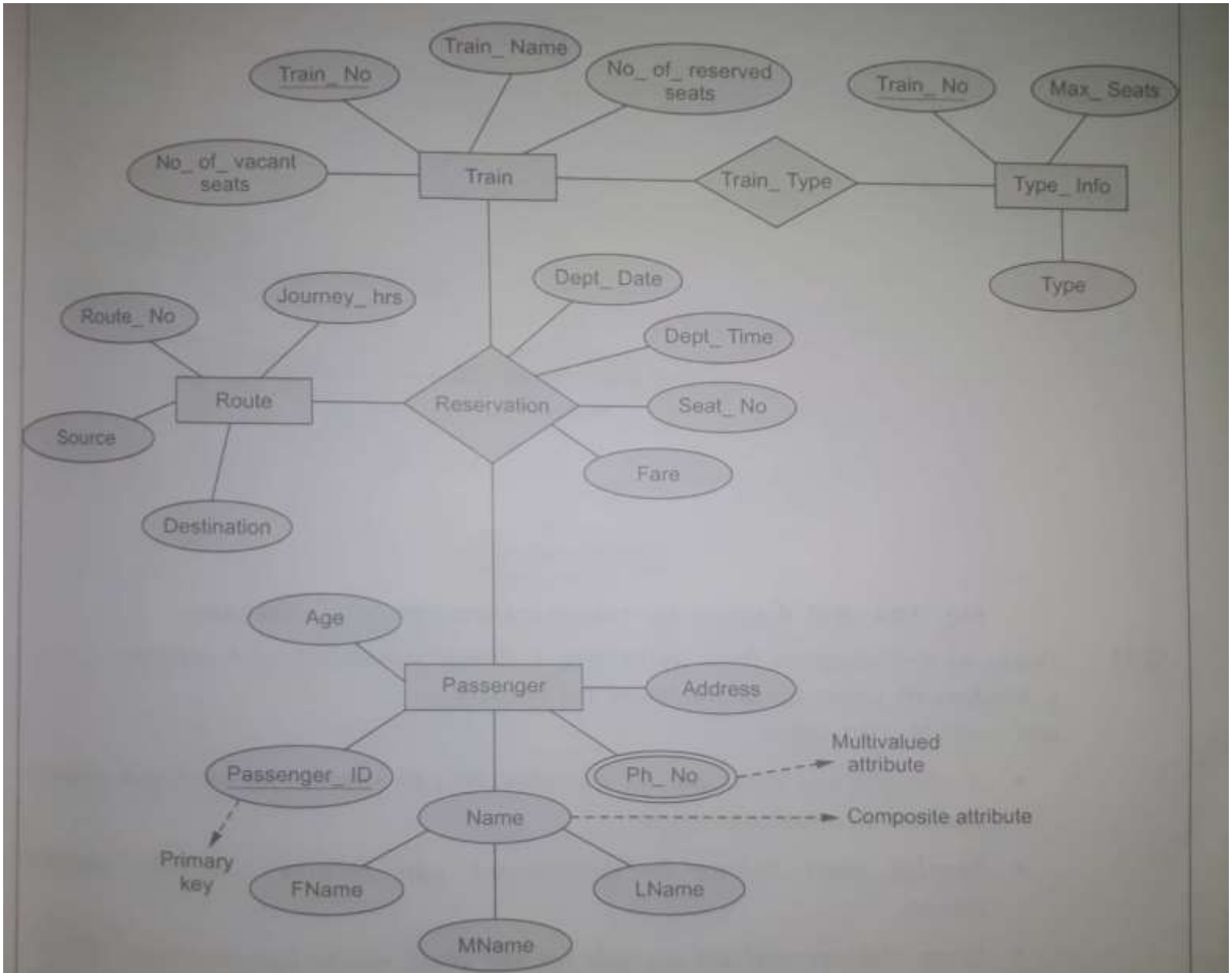
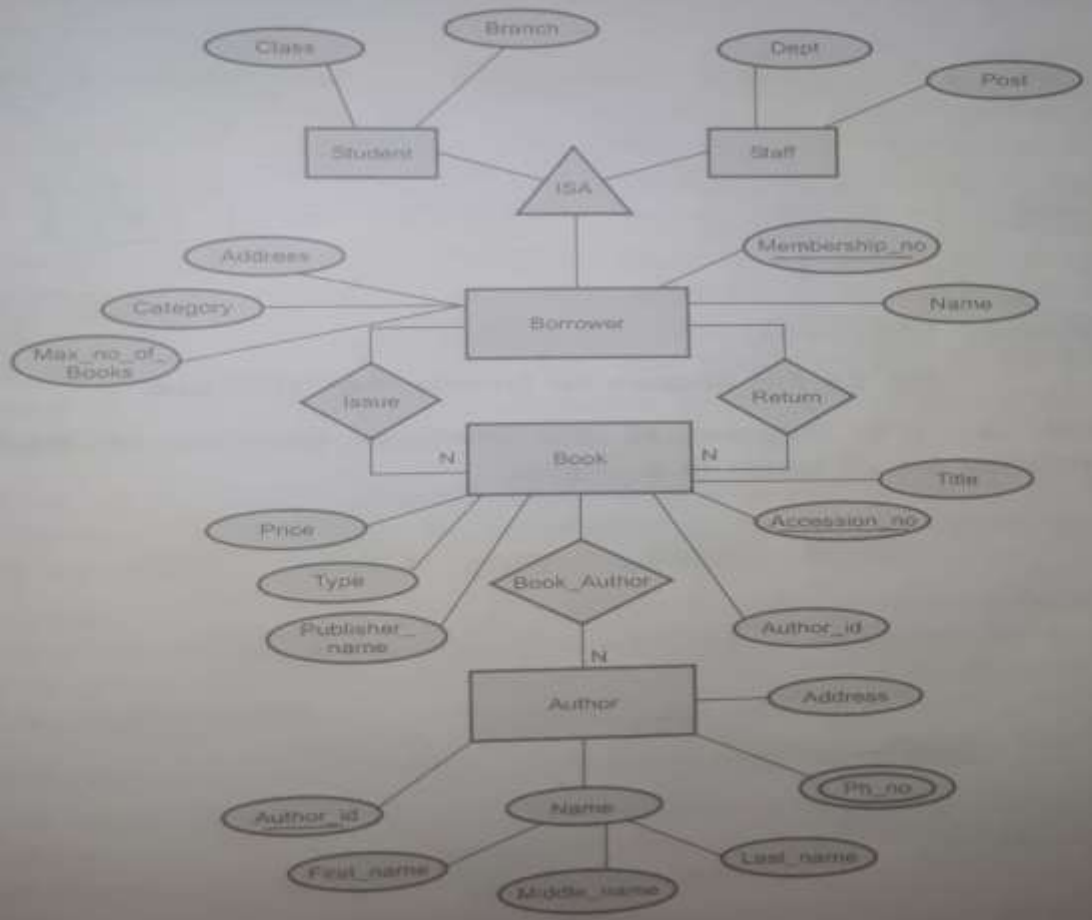


Fig. 2.13 E-R diagram for railway reservation system

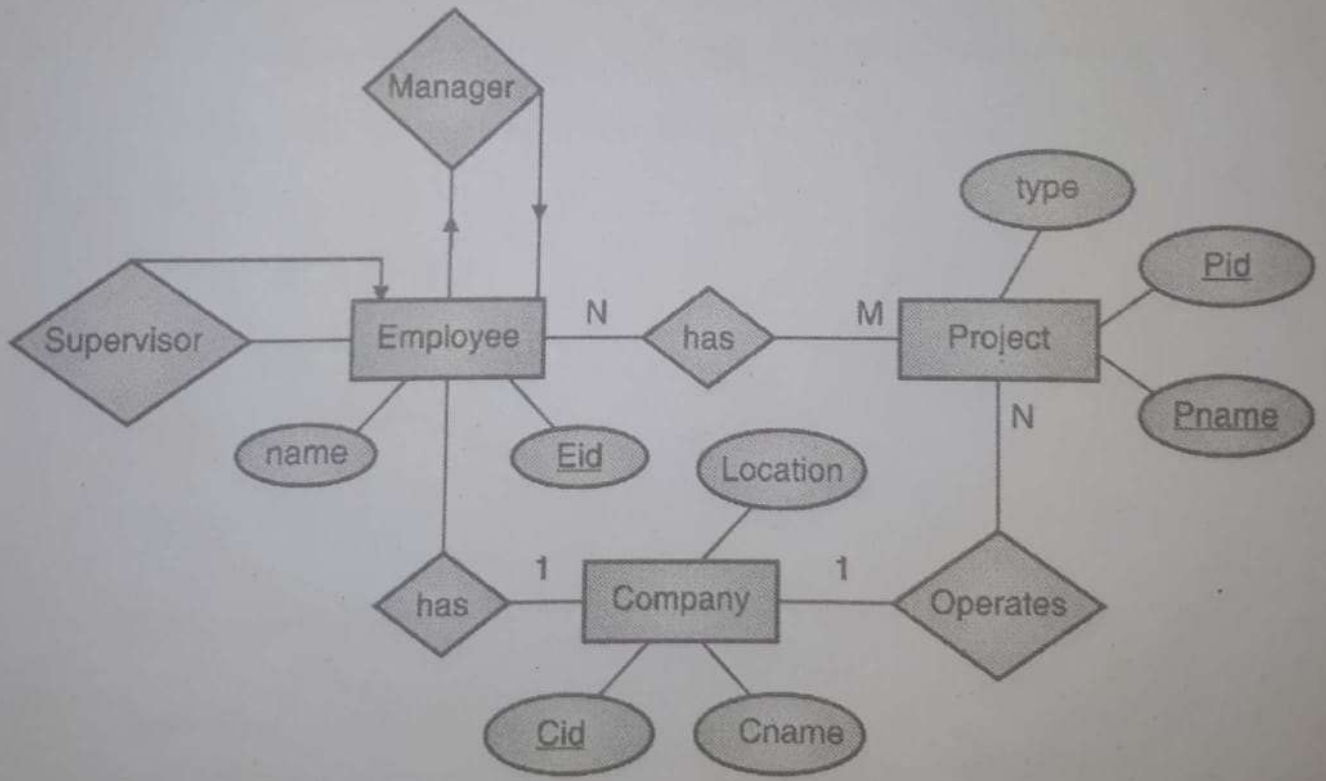
Following information is maintained in a library.
Books (Accession-no, Title author, price, book-type, publisher)
Borrower (Membership-no, Name, Address, Category, Max-no of books that can be issued, Accession-no of books borrowed)
The following constraints are to be observed.
i) Each book has unique accession-no.
ii) A book may have more than one author.
iii)The category of borrower determines the maximum number of books that may be issued to borrower.
iv) There may be more than one copy of a book.
Identify the entities, relationships and draw E-R diagram which provides for issue and return of a book, fine calculation and claiming of a issued book.



Example 4.12.4 : Construct ER diagram and convert into Relational Model for Company Which has several Employees working on different types of projects. Several Employees are working on one department. Every Employee has Manager. Several Employees are supervised by one Employee. **(10 Marks)**

Solution. :

1. Employee (Eid, Ename, mid, sup-id),Pid)
2. Company(Cid, Cname, Location)
3. Project (Pid, Pname, type)



(A) Training Course Company Case Study

- Consider a database for a training company that runs *training courses.*
- For this we must store data about the *trainees* and *instructors.*
- For each course participant we must record id, UID, name, age, employer's name, addr, telephone numbers, courses attended and final assessment for each course.
- We also need to represent the courses that each participant is attending at present and the place and time the classes are held.
- Each course has code and title and any course can be given any number of times.
- Each time a particular course is given we call it an *'edition'* of the course, and for each edition we represent the start date, end date and the number of participants.
- If a trainee is *self employed*, we need to know area of expertise, and other description.
- If a trainee is *employee* we record position, level and should also understand who is employer.
- For each instructor we record name, age, UID, qualification and the courses the instructor is qualified to teach.
- An instructor can be *employee of training company* or can be a *freelance.*

Steps

(1) Define Entity Types



Fig. 2.45 : Identified entities

(2) Identify Attributes

- These are identified as properties of chosen entities.
- These are attached to entities as shown in Fig. 2.46.

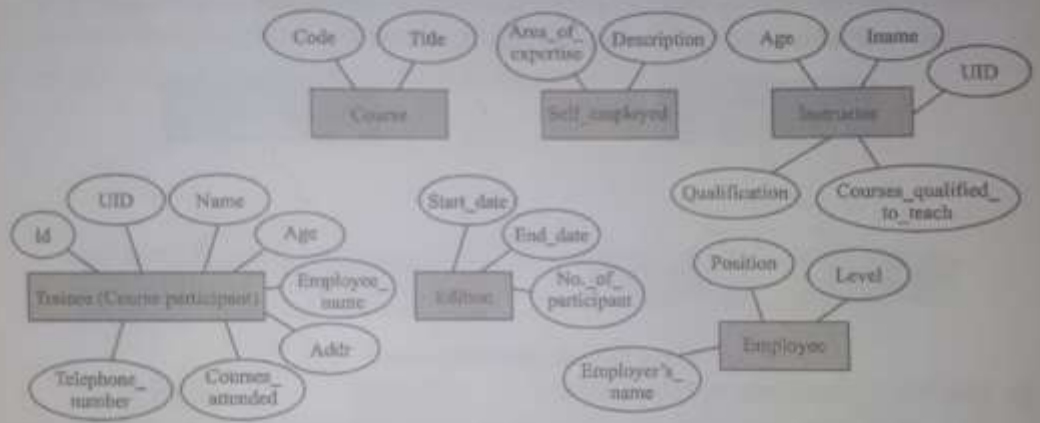


Fig. 2.46 : Attributes attached to entities

(3) Define Relationship Types

- These are usually verbs used in descriptions of the system.
- Relationships are attached between entities as shown in Fig. 2.47.

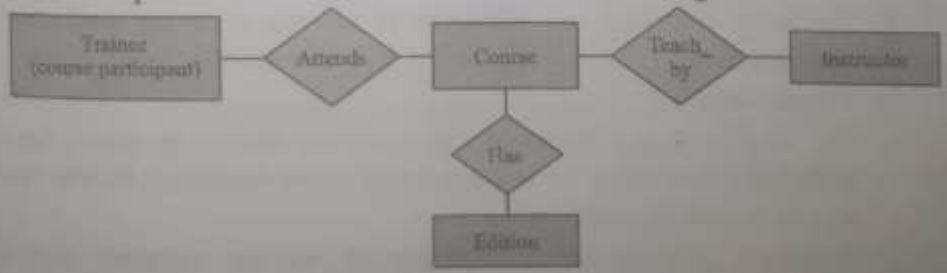


Fig. 2.47 : Partial ER diagram

(4) Add Cardinality and Participation to Relationships

- This deals with finding cardinality ratios, participation and weak entities.

- E.g. edition of a course is dependent on the course entity. Hence, edition is declared as weak entity having start date and end date combination acting as partial key (or discriminator).

(5) Identify Inheritance

- This deals with identifying specialized entities described in the problem statement.
- E.g. trainee can be self employed or employee which are described as specialized versions of trainee.
- Hence, trainee acts as a superclass while self employed and employee are its subclasses where they share all properties of their superclass.
- Other specializations (or Inheritances) are shown in Fig. 2.48.



Fig. 2.48 : Identified inheritance

(6) Identify Constraints on Inheritance

- This deals with identifying various constraints including type of specialization, disjointness and completeness constraint.

(a) Specialization Type

- Self employed and employee subclasses are attribute defined sub classes based on type of trainee.
- Freelance instructor and employee of training company instructor are attribute defined sub classes based on type of instructor.

(i) Disjointness

- Self employed and employee subclasses are disjoint in nature because participant can either be self employed or an employee. He/she cannot belong to both the subclasses.
- Freelance instructor and employee of training company instructor subclasses are disjoint in nature because participant can either be an employee of training company or a freelance. He/she cannot belong to both the subclasses.

(ii) Completeness

- Participation of type of trainee specialization will be partial because there might be some entities present in trainee entity set (superclass) which cannot be a part of any of the subclasses.
- Participation of type of instructor specialization will be total because all entities present in instructor entity set (superclass) will become part of one of the subclasses.

(7) Identify Union or Category

- When there arises a situation that needs the subclass to act as a subclass for only one superclass/subclass relationship even though there are more than one superclasses present, it has to be modeled as a union.
- Hence, final EER diagram is drawn as shown in Fig. 2.49.

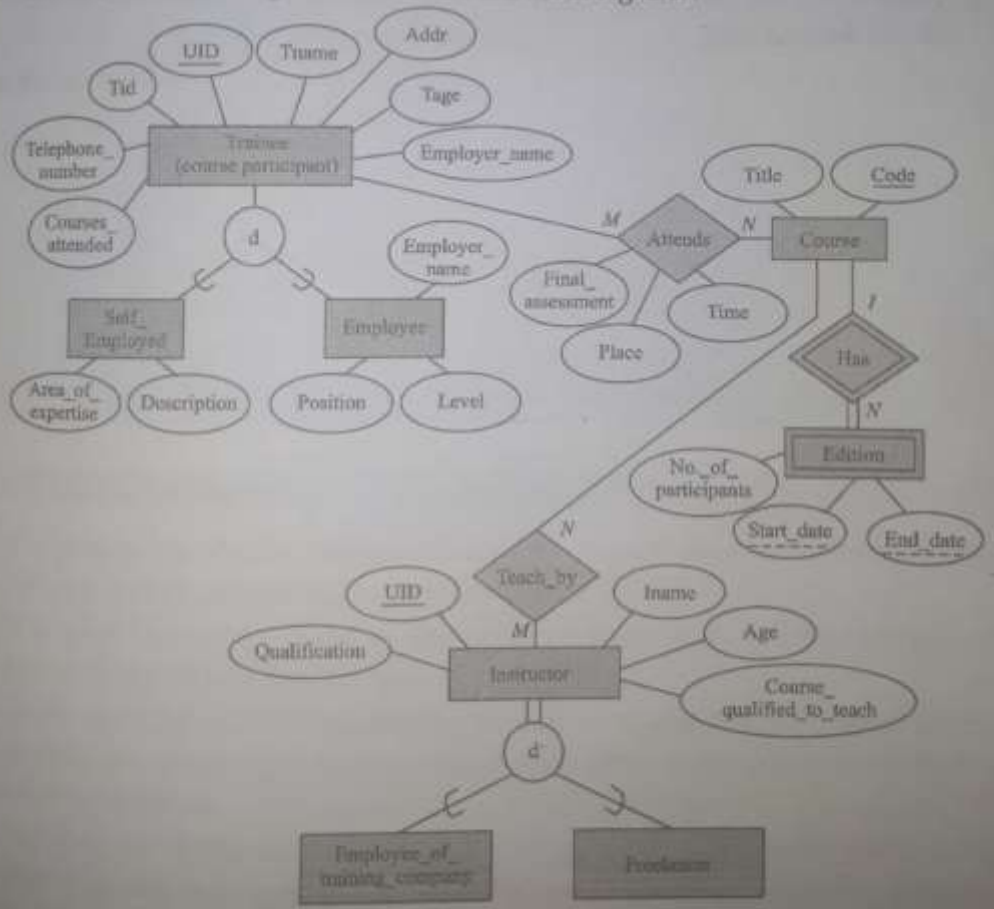


Fig. 2.49 : EER diagram for training company

SQL

Views in SQL

Views in SQL

- In SQL, a view is a virtual table based on the result-set of an SQL statement.

- A view contains rows and columns, just like a real table. The fields in a view are fields from one or more real tables in the database.

- A view can contain all rows of a table or selected rows from a table. A view can be created from one or many tables which depends on the written SQL query to create a view.

Views in SQL

- Views summarizes data from various tables which can be used to generate reports.

- To create the view, we can select the fields from one or more tables present in the database.

- A view can either have specific rows based on certain condition or all the rows of a table.

Views in SQL

- Views restrict access to the data because the view can display selective columns from the table.

- Views created with check option, prevents updating of other rows and columns.

- If a view is used to only look at table data and nothing else the view is called a Read-only view.

Views in SQL

- A view that is used to look at the table data as well as Insert, update and delete table data is called updatable view.

- Simple view: View derived from one table.

- Complex view: View derived from many tables (does not allow DML operations).

- **Views are very helpful in restricting access to your application's critical data to third-party users.**

We can create View using **CREATE VIEW** statement. A View can be created from a single table or multiple tables.

Syntax:

```
CREATE VIEW view_name AS
SELECT column1, column2.....
FROM table_name
WHERE condition;
```

view_name: Name for the View

table_name: Name of the table

condition: Condition to select rows

StudentDetails

S_ID	NAME	ADDRESS
1	Harsh	Kolkata
2	Ashish	Durgapur
3	Pratik	Delhi
4	Dhanraj	Bihar
5	Ram	Rajasthan

StudentMarks

ID	NAME	MARKS	AGE
1	Harsh	90	19
2	Suresh	50	20
3	Pratik	80	19
4	Dhanraj	95	21
5	Ram	85	18

- **Creating View from a single table:**
  - In this example we will create a View named DetailsView from the table StudentDetails.

    Query:

```
CREATE VIEW DetailsView AS
SELECT NAME, ADDRESS
FROM StudentDetails
WHERE S_ID < 5;
```

    To see the data in the View, we can query the view in the same manner as we query a table.

```
SELECT * FROM DetailsView;
```

    Output:

NAME	ADDRESS
Harsh	Kolkata
Ashish	Durgapur
Pratik	Delhi
Dhanraj	Bihar

- In this example, we will create a view named StudentNames from the table StudentDetails.
  Query:

```
CREATE VIEW StudentNames AS
SELECT S_ID, NAME
FROM StudentDetails
ORDER BY NAME;
```

If we now query the view as,

```
SELECT * FROM StudentNames;
```

Output:

S_ID	NAMES
2	Ashish
4	Dhanraj
1	Harsh
3	Pratik
5	Ram

- **Creating View from multiple tables**: In this example we will create a View named MarksView from two tables StudentDetails and StudentMarks. To create a View from multiple tables we can simply include multiple tables in the SELECT statement.

  Query:

```
CREATE VIEW MarksView AS
SELECT StudentDetails.NAME, StudentDetails.ADDRESS, StudentMarks.MARKS
FROM StudentDetails, StudentMarks
WHERE StudentDetails.NAME = StudentMarks.NAME;
```

  To display data of View MarksView:

```
SELECT * FROM MarksView;
```

  Output:

NAME	ADDRESS	MARKS
Harsh	Kolkata	90
Pratik	Delhi	80
Dhanraj	Bihar	95
Ram	Rajasthan	85

DELETING VIEWS

We have learned about creating a View, but what if a created View is not needed any more? Obviously we will want to delete it. SQL allows us to delete an existing View. We can delete or drop a View using the DROP statement.

Syntax:

```
DROP VIEW view_name;


view_name: Name of the View which we want to delete.
```

For example, if we want to delete the View **MarksView**, we can do this as:

```
DROP VIEW MarksView;
```

UPDATING VIEWS

There are certain conditions needed to be satisfied to update a view. If any one of these conditions is **not** met, then we will not be allowed to update the view.

1. The SELECT statement which is used to create the view should not include GROUP BY clause or ORDER BY clause.
2. The SELECT statement should not have the DISTINCT keyword.
3. The View should have all NOT NULL values.
4. The view should not be created using nested queries or complex queries.
5. The view should be created from a single table. If the view is created using multiple tables then we will not be allowed to update the view.

- **Inserting a row in a view**:

  We can insert a row in a View in a same way as we do in a table. We can use the INSERT INTO statement of SQL to insert a row in a View.**Syntax**:

```
INSERT INTO view_name(column1, column2 , column3,..)
VALUES(value1, value2, value3..);
```

view_name: Name of the View

Example:

In the below example we will insert a new row in the View DetailsView which we have created above in the example of "creating views from a single table".

```
INSERT INTO DetailsView(NAME, ADDRESS)
VALUES("Suresh","Gurgaon");
```

If we fetch all the data from DetailsView now as,

```
SELECT * FROM DetailsView;
```

Output:

NAME	ADDRESS
Harsh	Kolkata
Ashish	Durgapur
Pratik	Delhi
Dhanraj	Bihar
Suresh	Gurgaon

- **Deleting a row from a View**:

  Deleting rows from a view is also as simple as deleting rows from a table. We can use the DELETE statement of SQL to delete rows from a view. Also deleting a row from a view first delete the row from the actual table and the change is then reflected in the view.**Syntax**:

```
DELETE FROM view_name
WHERE condition;


view_name:Name of view from where we want to delete rows
condition: Condition to select rows
```

Example:

In this example we will delete the last row from the view DetailsView which we just added in the above example of inserting rows.

```
DELETE FROM DetailsView
WHERE NAME="Suresh";
```

If we fetch all the data from DetailsView now as,

```
SELECT * FROM DetailsView;
```

Output:

NAME	ADDRESS
Harsh	Kolkata
Ashish	Durgapur
Pratik	Delhi
Dhanraj	Bihar

Read only views

- If we don't want users to change data in base table or view, then in that case we can use the **with read only** clause.
- This clause will prevent user from making changes to base table.

For example

```
Create view emps-view as (select * from employees) with read only;
```

View created.
The next time if we try to select all attributes like

```
Select * from emps-view;
```

Then oracle returns the tables and generate the message.
6 rows selected.
But if we want to update emps-view then we write the SQL statement like

```
Update emps-view set ename = 'Ajay kumar' where ename = 'Ajay';
```

Then oracle returns the following message.
where ename = 'Ajay'
Error at line 2
ORA– 01733 Virtual column not allowed here.

•You can verify the list of all the views using the following query −

•SHOW FULL TABLES WHERE table_type = 'VIEW';

•Renaming Views in MySQL:

- RENAME TABLE original_view_name TO new_view_name;

WITH CHECK OPTION

Example:

In the below example we are creating a View SampleView from StudentDetails Table with WITH CHECK OPTION clause.

```
CREATE VIEW SampleView AS
SELECT S_ID, NAME
FROM  StudentDetails
WHERE NAME IS NOT NULL
WITH CHECK OPTION;
```

In this View if we now try to insert a new row with null value in the NAME column then it will give an error because the view is created with the condition for NAME column as NOT NULL.

For example,though the View is updatable but then also the below query for this View is not valid:

```
INSERT INTO SampleView(S_ID)
VALUES(6);
```

NOTE: The default value of NAME column is *null*.